arXiv:2605.11167v1 [cs.CL] 11 May 2026

The Bicameral Model: Bidirectional Hidden-State Coupling Between Parallel Language Models

Cedric Flamant* **Udaya Ghai** **Kanna Shimizu**
AWS Agentic AI

Abstract

Existing multi-model and tool-augmented systems communicate by generating text, serializing every exchange through the output vocabulary. Can two pretrained language models instead coordinate through a continuous, concurrent channel? The *Bicameral Model* couples two frozen language models through a trainable neural interface on their intermediate hidden states. At every generation step, both models run in lockstep: a primary model drives the task while an auxiliary model operates tools, solves constraints, or executes code, with both conditioning on each other's activations through a translation network and a learned suppression gate ($\sim$1% of combined parameters). The gate learns a selective communication protocol from task loss alone, without a prescribed format. We demonstrate the mechanism across three tool backends. On arithmetic, coupling two 0.5B models with a calculator raises accuracy from 36% to 96%. On logic grid puzzles, coupling two 0.6B models with a Z3 solver achieves $1.7\times$ the unaugmented baseline on ZebraLogic. On mathematical reasoning, coupling with a Python sandbox enables the auxiliary to generate problem-specific code from hidden-state signals alone, without ever seeing the problem text.

1 Introduction

Today, when two pretrained language models must cooperate, they communicate by generating text, serializing every exchange through discrete tokens. This paper asks whether two frozen LLMs can instead coordinate through their intermediate hidden states, and what the learned coupling looks like when trained only from task loss.

The Bicameral Model couples two language models through a trainable neural interface that provides a bridge for activations to flow between their intermediate hidden states. A primary model M_p orchestrates the task and produces the final response while an auxiliary model M_a maintains a concurrent stream with access to external tools. Both models advance in lockstep at every generation step, so the two streams condition on each other simultaneously and bidirectionally rather than through serialized text round-trips. The interface ϕ sits between configurable transformer layers, translating activations between the models' representation spaces and modulating the coupling strength through a learned suppression gate. Both models' weights remain frozen; only ϕ is trained via dual-target supervised fine-tuning.

The primary model continues generating while the auxiliary operates tools in parallel, with results delivered through the neural channel rather than a text round-trip. We test the mechanism across three tool backends chosen to create a large, unambiguous capability gap between primary and auxiliary: a calculator, the Z3 constraint solver, and a Python sandbox. Tools are the device we use to produce this gap, not the mechanism itself; the same hidden-state coupling should function for any pair of complementary models. Coupling a 0.5B model with a calculator-equipped auxiliary raises

*Correspondence to: cflamant@amazon.com

Preprint.

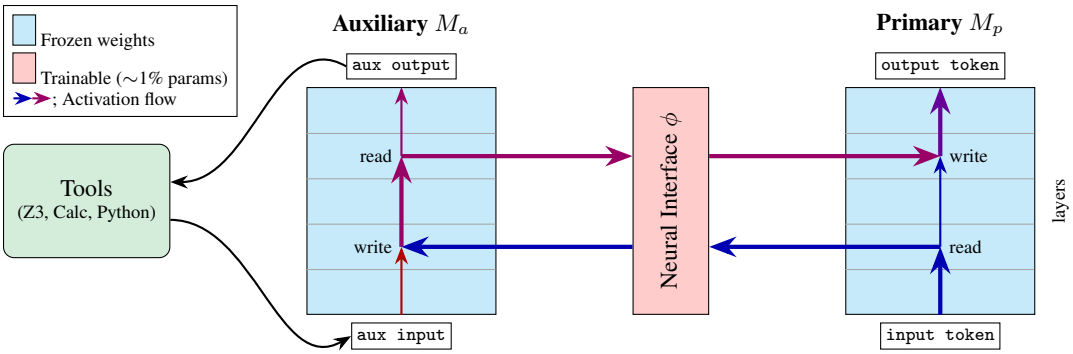


Figure 1: **Bicameral architecture.** A frozen primary model M_p and a frozen auxiliary model M_a run in parallel, coupled through a lightweight trainable neural interface ϕ. At each generation step, ϕ reads hidden states from M_p at layer ℓ_r^p and injects a perturbation into M_a at layer ℓ_w^a (forward coupling), then reads from M_a at layer ℓ_r^a and injects into M_p at layer ℓ_w^p (reverse coupling). The auxiliary model's output tokens are routed to external tools (Z3 solver, calculator); tool results are forced back as input tokens.

arithmetic accuracy from 36.2% to 96.5%; coupling a 0.6B model with a Z3 solver achieves a 1.7× improvement over the unaugmented model on ZebraLogic [Lin et al., 2025]; and in the Python-tool configuration, the auxiliary generates correct problem-specific code from hidden-state signals alone, without ever seeing the problem text.

Trained only on next-token loss, with no prescribed format for what the hidden-state channel should carry, the suppression gate converges to a selective, directionally structured communication protocol: forward coupling ($M_p \to M_a$) concentrates on task-relevant tokens such as numbers and operation keywords, while reverse coupling ($M_a \to M_p$) spikes when tool output becomes available or is recalled (§5). Moreover, this channel develops during training in a strict causal order, forward coupling first, then tool recall, then accuracy, consistent with the architecture's dependency structure. Adapter-equivalent ablations confirm the gains come from the auxiliary model's reasoning rather than from the trainable interface capacity.

2 Architecture

Notation. We use $m \in \{p, a\}$ to index the primary and auxiliary models. Each model is a pretrained transformer with L_m blocks; we write its residual-stream state before block $\ell \in \{0, 1, \ldots, L_m\}$ executes as $\mathbf{h}_m^{(\ell)}(t) \in \mathbb{R}^{d_m}$, where t is the token position. When $\ell = 0$ this is the input embedding, and when $\ell = L_m$ it is the state that feeds the final layer norm and unembedding; throughout the paper, "layer ℓ" refers to this residual-stream index. The tokens produced by each model at position t are denoted x_t^p and x_t^a (the primary and auxiliary tokens; sources described in §2.3). The interface reads and writes at four layer indices (read and write for each direction): $\ell_r^{p\to a}, \ell_w^{p\to a}, \ell_r^{a\to p}, \ell_w^{a\to p}$. Writing the forward direction for concreteness, the interface reads $\mathbf{h}_p^{(\ell_r^{p\to a})}(t)$ from M_p and replaces $\mathbf{h}_a^{(\ell_w^{p\to a})}(t)$ in M_a with a gated combination of the original state and a translated signal (§2.2); the reverse direction is symmetric. Our identity-coupling experiments constrain $\ell_r^{p\to a} = \ell_w^{p\to a}$ and $\ell_r^{a\to p} = \ell_w^{a\to p}$ (same-depth coupling, since identity requires $d_p = d_a$); PULLSTANDARD experiments sweep over general four-tuples.

2.1 System overview

The Bicameral Model consists of three components (Figure 1): a *primary model* M_p that generates user-facing text, an *auxiliary model* M_a that maintains a concurrent reasoning stream with access to external tools, and a *neural interface* that couples the two through their intermediate hidden states. Both M_p and M_a are pretrained transformer language models whose weights are frozen; only the parameters of the interface are trained.

The interface consists of two directional coupling operators, $\phi^{p\to a} : \mathbb{R}^{d_p} \times \mathbb{R}^{d_a} \to \mathbb{R}^{d_a}$ and $\phi^{a\to p} : \mathbb{R}^{d_a} \times \mathbb{R}^{d_p} \to \mathbb{R}^{d_p}$, each mapping a sender and receiver hidden state to an updated receiver state. Their concrete form (translation network plus learned suppression gate) is given in §2.2. A per-step forward pass then unfolds as (Figure 2):

1. Both models run from their input embeddings up to their forward-coupling layers, producing $\mathbf{h}_p^{(\ell_r^{p\to a})}(t)$ and $\mathbf{h}_a^{(\ell_w^{p\to a})}(t)$.
2. *Forward coupling* updates M_a's hidden state at $\ell_w^{p\to a}$:

$$\mathbf{h}_a^{(\ell_w^{p\to a})}(t) \;\leftarrow\; \phi^{p\to a}\Big(\mathbf{h}_p^{(\ell_r^{p\to a})}(t),\; \mathbf{h}_a^{(\ell_w^{p\to a})}(t)\Big). \tag{1}$$

3. Both models continue through the middle layers to the reverse-coupling layers, producing $\mathbf{h}_a^{(\ell_r^{a\to p})}(t)$ and $\mathbf{h}_p^{(\ell_w^{a\to p})}(t)$.
4. *Reverse coupling* updates M_p's hidden state at $\ell_w^{a\to p}$ by symmetry:

$$\mathbf{h}_p^{(\ell_w^{a\to p})}(t) \;\leftarrow\; \phi^{a\to p}\Big(\mathbf{h}_a^{(\ell_r^{a\to p})}(t),\; \mathbf{h}_p^{(\ell_w^{a\to p})}(t)\Big). \tag{2}$$

5. Both models complete their remaining layers and produce a token (sampled or forced; see Section 2.3).

Forward coupling lets the auxiliary observe the primary's evolving representation; reverse coupling lets the auxiliary's reasoning flow back and influence primary generation. We write $\phi = \{\phi^{p\to a}, \phi^{a\to p}\}$ for the interface as a whole; ϕ is the only trainable component, and M_p, M_a are frozen.

2.2 Neural interface

We now unpack the coupling operators $\phi^{p\to a}$ and $\phi^{a\to p}$ introduced in §2.1. Each direction has two trainable components: a *translation network* $f^{\cdot}$ that maps a sender hidden state into the receiver's representation space, and a *suppression gate* $g^{\cdot}$ that controls how much translated signal the receiver admits. Writing the forward direction for concreteness (reverse is symmetric):

$$\phi^{p\to a}(\mathbf{h}_p, \mathbf{h}_a) \;=\; \big(1 - \sigma^{p\to a}\big)\,\mathbf{h}_a \;+\; \sigma^{p\to a}\, f^{p\to a}(\mathbf{h}_p), \qquad \sigma^{p\to a} \;=\; \mathrm{Sigmoid}\big(g^{p\to a}(\mathbf{h}_a)\big). \tag{3}$$

Here $f^{p\to a} : \mathbb{R}^{d_p} \to \mathbb{R}^{d_a}$, and $g^{p\to a} : \mathbb{R}^{d_a} \to \mathbb{R}^{k}$ outputs a gate of dimension $k \in \{1, d_a\}$ (scalar or element-wise). The four networks $(f^{p\to a}, g^{p\to a}, f^{a\to p}, g^{a\to p})$ are the full parameterization of ϕ.

Pull design. The gate reads the *receiver's* hidden state, so the receiver decides how much external signal to admit based on its own state. Because Eq. 3 is a convex combination with $\sigma \in [0, 1]$, the two extremes have interpretable meanings: at $\sigma \approx 0$ the receiver ignores the sender and proceeds as if uncoupled; at $\sigma \approx 1$ the receiver's hidden state is replaced entirely by the translated sender signal.[2]

Instantiations. We consider two concrete choices of ϕ:

- **PULLSTANDARD**: each $f^{\cdot}$ is a multi-layer perceptron and each $g^{\cdot}$ is a smaller MLP, both with ReLU activations. Both scalar and element-wise gates are used across our sweep. This is the primary variant used in our experiments.
- **PULLIDENTITY**: $f^{\cdot}$ is replaced by the identity map (requiring $d_p = d_a$) and only the suppression gates are trained. This variant requires models with compatible latent spaces, e.g. the same model coupled at matching depths.

Layer adapters. In addition to ϕ, some configurations insert low-rank bottleneck adapters [Houlsby et al., 2019] directly on the frozen models' own transformer layers. These adapters are distinct from ϕ: each applies an additive residual $\mathbf{h} \leftarrow \mathbf{h} + \frac{\alpha}{r} W_{\mathrm{up}}(\mathrm{ReLU}(W_{\mathrm{down}}\mathbf{h}))$ to a layer's output, with W_{up} zero-initialized so the adapter is a no-op at initialization. We use them only with identity-style interfaces, where they appear necessary for the coupled models to handle a parallel stream of superposed hidden states.

[2] Preliminary experiments with "push" variants (gate computed from the sender) and gate-free variants also learn productive coupling; we report the pull design in what follows.

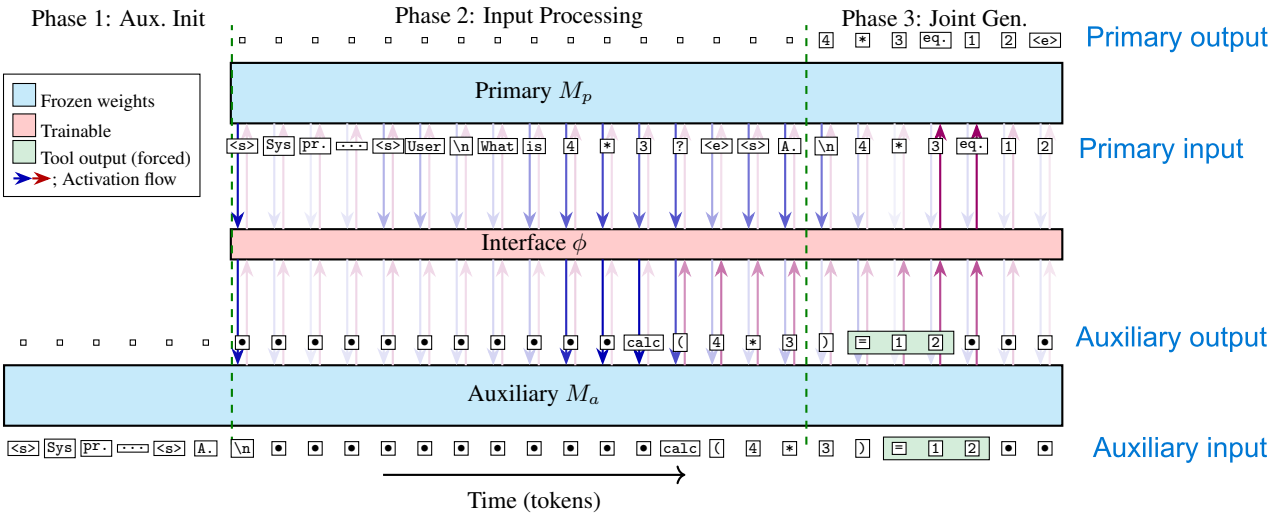


Figure 2: **Unrolled generation.** Token-by-token lockstep decoding for the arithmetic example "What is 4×3?" Arrow opacity reflects coupling strength (learned suppression gate σ). **Phase 1:** M_a processes its tool-instruction prompt (no coupling). **Phase 2:** M_p processes the user query; bidirectional coupling active ("wait" tokens • on M_a). **Phase 3:** Both models generate freely with bidirectional coupling. M_a emits `calc(4*3)`; the tool forces `=12` back as input tokens (green). M_p produces the answer "4 * 3 equals 12" without ever seeing calculator syntax.

2.3 Three-phase generation

At inference time, both models advance in lockstep one token per position. At step t each model emits one token: the primary token x_t^p and the auxiliary token x_t^a. Each token comes from one of three sources:

- **Sampled**, drawn from the model's conditional distribution with the coupling signals from ϕ already injected at the relevant hidden states (e.g. $x_t^p \sim M_p(\cdot \mid x_{<t}^p)$).
- **Input-forced** from a given prefix (system prompt, user query). The forward pass is still run so the KV cache and coupling remain well-defined, but no sampling occurs at that position on that stream.
- **Tool-forced** from an external tool (§2.4) that has been invoked earlier in the auxiliary stream. These tokens are not sampled from M_a and are masked from the auxiliary loss during training.

Bicameral generation proceeds in three phases (Figure 2), which differ only in these source rules:

- **Phase 1 (auxiliary prompt processing).** $x_t^a \leftarrow$ prompt; the primary stream is inactive. Coupling disabled; this phase just populates M_a's KV cache.
- **Phase 2 (primary input processing).** $x_t^p \leftarrow$ query; $x_t^a \sim M_a$ or $\leftarrow$ tool output. Full bidirectional coupling is active so that both models build representations of the query before joint generation begins.
- **Phase 3 (joint generation).** $x_t^p \sim M_p$; $x_t^a \sim M_a$ or $\leftarrow$ tool output. Full bidirectional coupling is active. Generation terminates when the primary emits an end-of-sequence token.

2.4 Tool integration

The auxiliary model interacts with external tools during generation. We demonstrate three backends:

Calculator. The auxiliary model generates expressions in the format `calc(expr)`. The expression is evaluated and the result is forced back as tokens `=result;` into the auxiliary stream. For example, `calc(564*848)` produces `=478272;`.

Z3 constraint solver. We define ZebraDSL, a custom constraint language for logic grid puzzles, backed by the Z3 SMT solver [de Moura and Bjørner, 2008]. The auxiliary model emits entity declarations, constraints, and queries; a pipeline (lexer, parser, semantic analyzer, Z3 translator)

processes them and returns solver results. For example, `bob.color=red;` assigns an attribute and `?bob.house;` queries the solver (Appendix C.4).

Python sandbox. The auxiliary model generates ```` ```python ```` code blocks. A subprocess-based sandbox executes the code and forces the output back as ```` ```output ```` blocks. In all cases, tool output tokens are forced into the auxiliary stream and masked from the auxiliary loss during training. The primary model never sees tool output in its token stream; it receives influence only through the hidden-state coupling.

3 Training

3.1 Dual-target supervised fine-tuning

Both language models are frozen. Training optimizes only the neural interface parameters θ_ϕ. The loss is the sum of masked cross-entropy losses on both models' outputs:

$$\mathcal{L}(\theta_\phi) = \mathcal{L}_p(\theta_\phi) + \mathcal{L}_a(\theta_\phi) \tag{4}$$

where $\mathcal{L}_p$ is computed over the primary model's response tokens and $\mathcal{L}_a$ is computed over the auxiliary model's generated content (forced tool outputs are masked).

Training uses teacher forcing on full sequences processed in parallel, just like regular SFT. The coupling architecture admits this because each model's forward pass decomposes around the coupling points: we run the pre-coupling layers of both models on their full input sequences in parallel, apply forward coupling to all hidden states at once, then run the post-coupling layers (including reverse coupling) up to the final token outputs.

3.2 Causality-constrained data alignment

The auxiliary model's actions must respect causality: it should not act on information the primary model has not yet produced at that token position. Similarly, if the primary model is trained to answer before it could have possibly received the tool output from the auxiliary model, it will never learn to use it. Training examples use a constraint solver to place auxiliary content (tool calls, reasoning steps) at valid positions relative to the primary token sequence. Causality tags mark the earliest and latest valid placement for each auxiliary content block, and a scheduling strategy (eager, lazy, random, or balanced) selects the final position within the valid window (Appendix C.10).

3.3 Reinforcement learning

The architecture also supports reinforcement learning as a second phase after SFT. We implement GRPO [Shao et al., 2024] over the interface parameters only (both LLMs remain frozen), using outcome-based reward with trajectories generated through the full coupled pipeline. Preliminary experiments on arithmetic show consistent improvements on held-out word problems (+4.7pp on GSM8K-IRL) with no regression on calculator-based arithmetic (Appendix C.11).

4 Experiments

We select domains where the tool creates an unambiguous capability gap, making the coupling's contribution easy to isolate from the base model's innate abilities. We evaluate the Bicameral Model in two domains: arithmetic with a calculator tool (Section 4.1) and logic grid puzzles with a Z3 constraint solver (Section 4.2).

4.1 Arithmetic with calculator

We couple two frozen copies of Qwen2.5-0.5B-Instruct [Yang et al., 2024] through a PULLSTANDARD interface, with only the auxiliary model having access to a calculator. We evaluate on general arithmetic, GSM8K [Cobbe et al., 2021], and GSM8K-IRL, a harder rephrasing of GSM8K we generated with professionally-themed contexts and decimal quantities. Full experimental details are in Appendix B.

Table 1: Arithmetic results (Qwen2.5-0.5B-Instruct). PULLSTANDARD bicameral row reports the best of 890 layer-wiring configurations; 95.4% beat the primary-only arithmetic baseline. Identity variants use same-depth coupling at layers 10/15. The adapter-equivalent bypasses the auxiliary model, feeding the forward network's output directly to the reverse network.

Configuration	Params	Arith.	GSM8K	GSM8K-IRL
Primary-only (no interface)	—	36.2	**49.6**	12.8
*Learned translation (*PULLSTANDARD*)*				
Adapter-equivalent	16M	48.0	40.9	9.2
Bicameral (best of 890)	16M	**96.5**	39.1	17.1
Identity coupling (same-depth, layers 10/15)				
ELEMIDENTITY + adapters	919K	84.2	40.8	**22.2**
SCALARIDENTITY + adapters[†]	803K	68.1±11.8	36.8±1.5	17.5±3.0
SCALARIDENTITY, no adapters	115K	39.0	42.8	11.1

[†]Mean ± std over 5 seeds. Other rows are single runs.

Main results. We sweep 890 sampled configurations, varying which coupling layers are selected. Table 1 reports the best configuration alongside baselines. Of the 890 configurations, 95.4% beat the primary-only arithmetic baseline, indicating robustness to layer choice. On the harder GSM8K-IRL benchmark, 23.7% of configurations improve over the primary-only baseline (12.8%), with the best reaching 17.1% (+4.3pp).[3] On standard GSM8K, no configuration matches the primary-only baseline (49.6%), likely because hidden-state perturbations disrupt reasoning patterns the base model already executes well and the calculator adds little value on simple arithmetic.

Adapter-equivalent ablation. To test whether the gains come from the auxiliary model's calculator or merely from the additional 16M trainable parameters, we train an adapter-equivalent configuration (Appendix C.6): the same interface and training procedure, but with M_a bypassed so signals round-trip through ϕ alone ($M_p \to \phi \to M_p$). The adapter-equivalent scores 48.0% on arithmetic (versus 96.5% bicameral) and 9.2% on GSM8K-IRL (versus 17.1%), confirming that the auxiliary model's tool use, not the parameter count, drives the gains.

Identity coupling. An alternative approach is PULLIDENTITY: hidden states pass directly between models with only a suppression gate and low-rank adapters. With ∼800K–900K trainable parameters, identity interfaces sacrifice peak arithmetic accuracy (68–84% vs 96.5%) but *outperform* learned translation on word problems, reaching 22.2% on GSM8K-IRL (9.4pp above primary-only, 5.1pp above PULLSTANDARD). Operating directly in the models' native representation space without compressing through a learned bottleneck may be what helps generalization to out-of-distribution problems.

4.2 Logic puzzles with Z3 solver

We couple two frozen copies of Qwen3-0.6B through a PULLSTANDARD interface. The auxiliary emits ZebraDSL, a small declarative language we designed for logic grid puzzles (e.g., discrete-choice assignment puzzles such as Sudoku or clue-based logic grids). ZebraDSL compiles to assertions for the Z3 SMT solver [de Moura and Bjørner, 2008], which checks whether the constraints are consistent and returns any implied values. We evaluate on ZebraLogic [Lin et al., 2025], a benchmark of 1,000 logic grid puzzles, and on GeneralZebra, a dynamically generated in-distribution test suite. Full details and the complete ZebraDSL specification are in Appendices B and C.4.

Main results. Table 2 presents four training configurations that vary in clue presentation style. Our best configuration (40 epochs, ZL-style clues) reaches $64.7 \pm 6.7\%$ on ZebraLogic (mean ± std over 5 seeds), a $1.7\times$ improvement over the unaugmented Qwen3-0.6B baseline (37.5% with thinking, 32k max tokens). For context, the ZebraLogic leaderboard reports 36.2% for Claude 3.5 Sonnet and 31.7% for GPT-4o, though these models lack solver access. Training without ZebraLogic-style clue presentation drops ZebraLogic accuracy to 10.3% despite 81.1% on GeneralZebra, indicating the auxiliary leans heavily on memorized clue patterns; the text-level tool baseline below suggests this

[3]The sweep includes configurations expected to underperform, such as shallow coupling and wirings without a within-token return path, to map the full design space; see Appendix C.15.

Table 2: ZebraLogic and GeneralZebra results. Primary and auxiliary models are both Qwen3-0.6B [Yang et al., 2025]. All bicameral configurations use PULLSTANDARD with 21M interface parameters. Primary-only baselines evaluated at 32k tokens. Adapter-equivalent uses the same interface but bypasses the auxiliary model.

Configuration	ZebraLogic	GeneralZebra
Primary-only (thinking, 32k)	37.5	15.2
Primary-only (no thinking, 32k)	4.9	2.9
Adapter-equivalent (40 ep.)	7.5	7.9
Bicameral, w/o ZL clue style (20 ep.)	10.3	81.1
Bicameral, w/ ZL clue style (20 ep.)	48.3	87.0
Bicameral, ZL clue style only (20 ep.)	45.3	8.0
Bicameral, w/ ZL clue style (40 ep.)	**64.7** $\pm$ **6.7**	**93.2** $\pm$ **0.9**

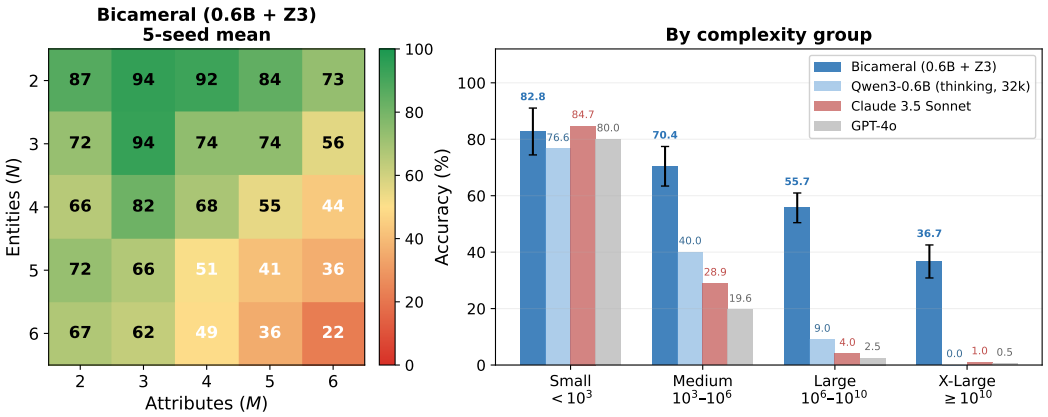


Figure 3: ZebraLogic accuracy by puzzle complexity (5-seed mean). **Left:** Per-size heatmap ($n = 40$ per cell per seed). **Right:** Grouped accuracy with seed std error bars; Claude 3.5 Sonnet and GPT-4o baselines from Lin et al. [2025] shown for calibration (these models lack solver access).

rigidity reflects a Qwen3-0.6B capability ceiling rather than the architecture, since the model cannot use ZebraDSL as text at all without training. As a preliminary probe of entity generalization, a single attempt at replacing ZebraLogic entities with values absent from all training categories produced a correct solution (Figure 9); systematic OOD entity evaluation is left to future work.

Accuracy by puzzle size. Figure 3 reports accuracy per puzzle size, grouped by ZebraLogic's search-space bins [Lin et al., 2025]. The bicameral system reaches $82.8 \pm 8.3\%$ on small puzzles (search space $< 10^3$) and drops to $36.7 \pm 5.8\%$ on extra-large puzzles (search space $\geq 10^{10}$); the Claude 3.5 Sonnet and GPT-4o curves are plotted alongside for calibration, with the gap widening sharply as search space grows.

Text-level tool baseline. We prompt thinking-enabled Qwen3-0.6B with ZebraDSL syntax instructions and pipe its text output through the ZebraDSL execution pipeline. This baseline scores 24.6% (below the 37.5% primary-only baseline) with zero successful Z3 queries across all 1,000 puzzles: all correct answers come from chain-of-thought, not the solver. Tool instructions alone do not enable Z3 use at this model size, though dedicated SFT would likely close this gap.

5 Analysis and discussion

Coordination develops as a phase transition. To understand *how* the coupling develops, we train on multiplication-only data with inputs uniformly sampled over the range $[1, 10^7]$ and track four metrics: forward perturbation norm, reverse perturbation norm, tool recall (fraction of problems where the auxiliary calls the calculator with correct operands), and task accuracy (Figure 4). Two scalar-gated architectures are compared: the gated identity interface and the gated MLP interface. Both scalar gates output initially near zero, so coupling strength starts negligible and must be actively learned.

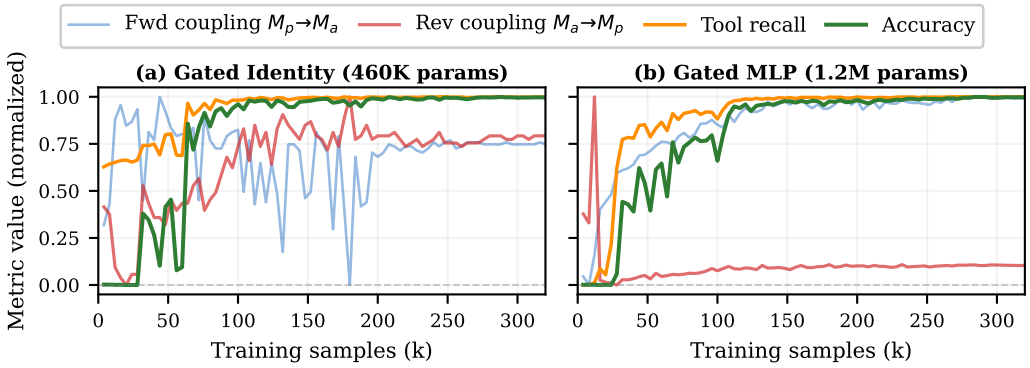


Figure 4: **Communication onset during training.** Normalized coupling strength, tool recall, and accuracy over 320k training samples (multiplication-only, $[1, 10^7]$, primary-only baseline 0%) for (a) Gated Identity (SCALARIDENTITY, 460K params) and (b) Gated MLP (PULLSTANDARD, 1.2M params). Forward coupling activates immediately; tool recall develops next; accuracy undergoes sudden onset once tool recall crosses a threshold. Both architectures reach 99.9% accuracy.

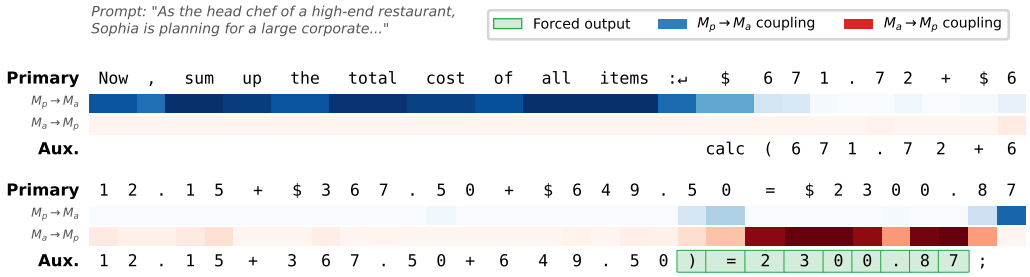


Figure 5: **Coupling activity during arithmetic generation.** Token-by-token coupling strength for a multi-step word problem (Qwen2.5-0.5B, scalar identity interface, tokens 464–515). **Top two rows:** Primary model tokens with forward (blue) and reverse (red) coupling strength. **Bottom two rows:** Auxiliary model tokens with the same coupling channels. Forward coupling activates on relevant primary tokens ("sum," "total," "all items") but not on the dollar amounts themselves, because the auxiliary has already received those values earlier in the sequence. Reverse coupling is near zero until the calculator returns its result (`=2300.87`), at which point it spikes as the answer flows back to the primary model. See Figure 8 for the full trace.

Both architectures show the same qualitative pattern. Forward coupling activates immediately (within 4k samples), establishing a channel from primary to auxiliary. Tool recall ramps next: the gated identity starts at 63% (compatible raw hidden states already carry useful signal) and climbs to 97% by 64k samples; the gated MLP starts at 0% (the translation must be learned from scratch) and reaches 85% by 32k samples. Accuracy undergoes a sudden onset once tool recall crosses a threshold: the system spends 28k samples at 0% then jumps to 40–60% within 4–8k samples, stabilizing above 95% by 100–120k samples. Both architectures reach 99.9%.

Selective coupling. The suppression gate learns a structured activation pattern from task loss alone, with no prescribed protocol. Figure 5 shows the coupling strength at each token position during a multi-step arithmetic problem. Forward coupling ($M_p \to M_a$, blue) activates on relevant tokens: "sum," "total," "cost," and "all items". Notably, the forward channel does not activate on the dollar amounts themselves because the auxiliary already received those values earlier in the sequence and does not need them repeated. Reverse coupling ($M_a \to M_p$, red) is near zero throughout this period and spikes only when the calculator returns its result (`=2300.87`).

Interface capacity trades in-distribution fit for OOD generalization. PULLSTANDARD's learned translation achieves the highest in-distribution accuracy (96.5% arithmetic) but is outperformed by

identity coupling on out-of-distribution GSM8K-IRL (22.2% vs 17.1%, Section 4.1). The pattern is consistent with a capacity–generalization tradeoff: a freely learned translation fits the training distribution's exact representational structure but transfers less reliably than a near-identity pass-through with a learned gate, which leans on the inductive bias that two copies of the same pretrained model have compatible representations at matching depths.

Mathematical reasoning with Python sandbox. Coupling twin Qwen3-4B models with a Python sandbox on NuminaMath-TIR [AI-MO/NuminaMath-TIR contributors, 2024] yields 62.5% on MATH [Hendrycks et al., 2021], below the unaugmented Qwen3-4B-with-thinking baseline (81.6% at 32k tokens). However, the coupling produces correct answers on 127 problems where thinking alone fails, and qualitative inspection of the hidden-state channel shows problem-specific code being generated (Appendices C.12, C.19).

Limitations. Lockstep generation can double compute requirements. The coupling degrades performance on tasks the base model already handles well: GSM8K drops from 49.6% to around 40% across all 890 configurations, consistent with the framing that when the capability gap is small, hidden-state perturbations inject more noise than signal. Training also requires task-specific data with causality annotations (§7 discusses a possible RL relaxation).

6 Related work

Latent-space communication between LLMs. A growing body of work replaces text-based inter-model communication with hidden-state transfer. Ramesh and Li [2025] show that even a zero-parameter one-shot activation graft between frozen models improves over natural-language debate, but their coupling is unidirectional and applied once rather than at every generation step. C2C [Fu et al., 2026] fuses one model's KV-cache into another's via learned projections with per-layer Gumbel-sigmoid gating, again unidirectional and applied once at prefill. Interlat [Du et al., 2026] trains a communication adapter for latent transfer from a frozen sender to a fine-tuned receiver in a turn-based setup. LatentMAS [Zou et al., 2025] transfers full KV caches between agents in a training-free sequential pipeline. CALM [Bansal et al., 2024] composes two frozen models via cross-attention on intermediate representations. Concurrent work by Yang et al. [2026] replaces text-based multi-agent communication with latent-state transfer through trainable projections between frozen LLMs in a recursive pipeline, but their agents communicate in sequential discrete rounds. The Bicameral Model differs from all of these in providing *bidirectional continuous coupling* at every decoding step.

Multi-agent systems. Debate [Du et al., 2024], self-consistency [Wang et al., 2023], and mixture-of-agents communicate through generated text. ThoughtComm [Zheng et al., 2025] enhances multi-agent debate by injecting latent thoughts from agents' hidden states as prefix embeddings between rounds. These approaches are turn-based; bicameral coupling instead operates at every token step.

Representation alignment. The Platonic Representation Hypothesis [Huh et al., 2024] argues that model representations converge across architectures, supporting the feasibility of learned projections between LLM latent spaces. FuseChat [Wan et al., 2024] and Co-LLM [Shen et al., 2024] compose models at the output level, not via intermediate hidden states.

7 Conclusion

Two frozen language models connected by a lightweight neural interface on their hidden states learn to cooperate on tasks neither can solve alone, across arithmetic, logic puzzles, and Python-assisted math. The communication is selective: the learned suppression gate fires on task-relevant tokens and stays silent otherwise, producing a structured protocol without a prescribed format; adapter-equivalent ablations confirm the gains come from the auxiliary's reasoning, not the interface parameters. Hidden-state coupling opens a design space for multi-model systems that communicate below the token level: the architecture should extend to other complementary-model pairings such as retrieval or non-text-modality models, and, with adaptations to the training objective, to reward or process-reward models delivering dense quality signal. A second direction is leaning further into RL on the interface (§3.3): scaling to more domains and larger models, and relaxing SFT's causality-annotation requirements as RL picks up the coordination timing.

References

AI-MO/NuminaMath-TIR contributors. NuminaMath-TIR. HuggingFace Datasets: `AI-MO/NuminaMath-TIR`, 2024.

Anthropic. Claude 3.5 sonnet model card addendum. `https://www-cdn.anthropic.com/fed9cc193a14b84131812372d8d5857f8f304c52/Model_Card_Claude_3_Addendum.pdf`, 2024.

Rachit Bansal, Bidisha Samanta, Siddharth Dalmia, Nitish Gupta, Shikhar Vashishth, Sriram Ganapathy, Abhishek Bapna, Prateek Jain, and Partha Talukdar. LLM augmented LLMs: Expanding capabilities through composition. *arXiv preprint arXiv:2401.02412*, 2024.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

Hoagy Cunningham, Aidan Ewart, Logan Riggs, Robert Huben, and Lee Sharkey. Sparse autoencoders find highly interpretable features in language models. In *International Conference on Learning Representations (ICLR)*, 2024.

Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. Springer, 2008.

Varun Dhanraj and Chris Eliasmith. Improving rule-based reasoning in LLMs using neurosymbolic representations. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.

Harry Dong, David Brandfonbrener, Eryk Helenowski, Yun He, Mrinal Kumar, Han Fang, Yuejie Chi, and Karthik Abinav Sankararaman. Generalized parallel scaling with interdependent generations. *arXiv preprint arXiv:2510.01143*, 2025.

Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate. In *International Conference on Machine Learning (ICML)*, 2024.

Zhuoyun Du, Runze Wang, Huiyu Bai, Zouying Cao, Xiaoyong Zhu, Yu Cheng, Bo Zheng, Wei Chen, and Haochao Ying. Enabling agents to communicate entirely in latent space. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. In *Journal of Machine Learning Research (JMLR)*, 2022.

Tianyu Fu, Zihan Min, Hanling Zhang, Jichao Yan, Guohao Dai, Wanli Ouyang, and Yu Wang. Cache-to-cache: Direct semantic communication between large language models. In *International Conference on Learning Representations (ICLR)*, 2026.

Leo Gao, Tom Dupré la Tour, Henk Tillman, Gabriel Goh, Rajan Troll, Alec Radford, Ilya Sutskever, Jan Leike, and Jeffrey Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *International Conference on Machine Learning (ICML)*, 2023.

Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. ToRA: A tool-integrated reasoning agent for mathematical problem solving. In *International Conference on Learning Representations (ICLR)*, 2024.

Shibo Hao, Tianyang Liu, Zhen Wang, and Zhiting Hu. ToolkenGPT: Augmenting frozen language models with massive tools via tool embeddings. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. In *Conference on Language Modeling (COLM)*, 2025.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for NLP. In *International Conference on Machine Learning*, 2019.

Minyoung Huh, Brian Cheung, Tongzhou Wang, and Phillip Isola. The platonic representation hypothesis. *arXiv preprint arXiv:2405.07987*, 2024.

Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. In *International Conference on Learning Representations (ICLR)*, 2023.

Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024.

Kenneth Li, Oam Patel, Fernanda Viégas, Hanspeter Pfister, and Martin Wattenberg. Inference-time intervention: Eliciting truthful answers from a language model. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Bill Yuchen Lin, Ronan Le Bras, Kyle Richardson, Ashish Sabharwal, Radha Poovendran, Peter Clark, and Yejin Choi. Zebralogic: On the scaling limits of LLMs for logical reasoning. In *International Conference on Machine Learning (ICML)*, 2025.

Samuel Marks, Can Rager, Eric J. Michaud, Yonatan Belinkov, David Bau, and Aaron Mueller. Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. In *International Conference on Learning Representations (ICLR)*, 2025.

Theo X. Olausson, Alex Gu, Benjamin Lipkin, Cedegao E. Zhang, Armando Solar-Lezama, Joshua B. Tenenbaum, and Roger Levy. LINC: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

Liangming Pan, Alon Albalak, Xinyi Wang, and William Yang Wang. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. In *Findings of the Association for Computational Linguistics: EMNLP*, 2023.

Vignav Ramesh and Kenneth Li. Communicating activations between language model agents. In *International Conference on Machine Learning (ICML)*, 2025.

Nina Rimsky, Nick Gabrieli, Julian Schulz, Meg Tong, Evan Hubinger, and Alexander Turner. Steering Llama 2 via contrastive activation addition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Shannon Zejiang Shen, Hunter Lang, Bailin Wang, Yoon Kim, and David Sontag. Co-LLM: Learning to decode collaboratively with multiple language models. *arXiv preprint arXiv:2403.03870*, 2024.

Adly Templeton, Tom Conerly, Jonathan Marcus, et al. Scaling monosemanticity: Extracting interpretable features from Claude 3 Sonnet. Transformer Circuits Thread, Anthropic, 2024. https://transformer-circuits.pub/2024/scaling-monosemanticity/.

Eric Todd, Millicent L. Li, Arnab Sen Sharma, Aaron Mueller, Byron C. Wallace, and David Bau. Function vectors in large language models. In *International Conference on Learning Representations (ICLR)*, 2024.

Alexander Matt Turner, Lisa Thiergart, Gavin Leech, David Udell, Juan J. Vazquez, Ulisse Mini, and Monte MacDiarmid. Steering language models with activation engineering. *arXiv preprint arXiv:2308.10248*, 2023.

Fanqi Wan, Longguang Zhong, Ziyi Yang, Ruijun Chen, and Xiaojun Quan. FuseChat: Knowledge fusion of chat models. *arXiv preprint arXiv:2408.07990*, 2024.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Mitchell Wortsman, Gabriel Ilharco, Samir Yitzhak Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S. Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, and Ludwig Schmidt. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning (ICML)*, 2022.

Shiguang Wu, Yaqing Wang, and Quanming Yao. Dense communication between language models. *arXiv preprint arXiv:2505.12741*, 2025.

Prateek Yadav, Derek Tam, Leshem Choshen, Colin Raffel, and Mohit Bansal. TIES-Merging: Resolving interference when merging models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, et al. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

Xiyuan Yang, Jiaru Zou, Rui Pan, Ruizhong Qiu, Pan Lu, Shizhe Diao, Jindong Jiang, Hanghang Tong, Tong Zhang, Markus J. Buehler, Jingrui He, and James Zou. Recursive multi-agent systems. *arXiv preprint arXiv:2604.25917*, 2026.

Xi Ye, Qiaochu Chen, Isil Dillig, and Greg Durrett. SatLM: Satisfiability-aided language models using declarative prompting. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super Mario: Absorbing abilities from homologous models as a free lunch. In *International Conference on Machine Learning (ICML)*, 2024.

Yujia Zheng, Zhuokai Zhao, Zijian Li, Yaqi Xie, Mingze Gao, Lizhu Zhang, and Kun Zhang. Thought communication in multiagent collaboration. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

Andy Zou, Long Phan, Sarah Chen, James Campbell, Phillip Guo, Richard Ren, Alexander Pan, Xuwang Yin, Mantas Mazeika, Ann-Kathrin Dombrowski, Shashwat Goel, Nathaniel Li, Michael J. Byun, Zifan Wang, Alex Mallen, Steven Basart, Sanmi Koyejo, Dawn Song, Matt Fredrikson, J. Zico Kolter, and Dan Hendrycks. Representation engineering: A top-down approach to AI transparency. *arXiv preprint arXiv:2310.01405*, 2023.

Jiaru Zou, Xiyuan Yang, Ruizhong Qiu, Gaotang Li, Katherine Tieu, Pan Lu, Ke Shen, Hanghang Tong, Yejin Choi, Jingrui He, James Zou, Mengdi Wang, and Ling Yang. Latent collaboration in multi-agent systems. *arXiv preprint arXiv:2511.20639*, 2025.

A Extended related work

Tool-augmented LLMs. Toolformer [Schick et al., 2023], ToRA [Gou et al., 2024], and PAL [Gao et al., 2023] teach a single model to produce tool-calling syntax, modifying its weights. ToolkenGPT [Hao et al., 2023] represents tools as learned token embeddings, enabling tool invocation through next-token prediction with frozen weights. The Bicameral Model takes a different approach: rather than embedding tool knowledge in one model, it routes tool access through a separate auxiliary model communicating via hidden states, keeping the primary model entirely unaware of tool syntax.

Neurosymbolic reasoning. Logic-LM [Pan et al., 2023], SatLM [Ye et al., 2023], and LINC [Olausson et al., 2023] interface language models with formal solvers through text-level declarative specifications. LINC demonstrates that a 15B model with a first-order logic prover can surpass GPT-4 with chain-of-thought [Wei et al., 2022], the same pattern we observe with our 0.6B model and Z3. Closer to our framing, Dhanraj and Eliasmith [2025] embed an LM's activations into a structured neurosymbolic vector space so rule-based steps happen in latent symbolic form before being decoded back to tokens. Our approach differs from text-level solver interfaces in routing solver interaction through a hidden-state channel rather than text, and differs from in-LM neurosymbolic subspaces in pushing the symbolic component into a separate model (the auxiliary emits Z3 constraints).

Hidden-state-level communication and coordination. Two recent lines of work share activations at the hidden-state level. Wu et al. [2025] strip the token-level input/output layers from multiple LMs and connect them through trainable dense-communication modules, producing an end-to-end-differentiable graph of LMs that matches monolithic LM performance at a fraction of the training cost; the Bicameral Model preserves each model's token-level IO so the primary's output remains language, and couples per-token through a learned gate rather than Wu et al.'s continuous differentiable stream. Dong et al. [2025] share intermediate activations across parallel generations of the *same* model via a learned bridge, turning independent samples into interdependent decoding paths (relatedly, Coconut [Hao et al., 2025] reasons in continuous latent space via hidden-state self-recurrence within one model); our coupling shares activations across models with complementary capabilities, while theirs shares activations across siblings for diversity.

Activation steering and representation engineering. A parallel line of work edits LM behavior by adding fixed directions to intermediate hidden states. Representation engineering [Zou et al., 2023] extracts concept directions that causally steer model outputs; function vectors [Todd et al., 2024] isolate task-specific directions that transfer across prompts; and contrastive activation addition [Rimsky et al., 2024], inference-time intervention [Li et al., 2023], and activation addition [Turner et al., 2023] intervene at inference with a static offset. The Bicameral Model's suppression gate can be viewed as a *learned, token-level, bidirectional* relative of these static-direction steerings: the gate decides per-token whether and how much sender-derived signal to admit, rather than injecting a fixed vector uniformly across a sequence.

Sparse autoencoders and feature circuits. A recent line of interpretability work trains sparse autoencoders on LM activations to decompose hidden states into monosemantic features [Cunningham et al., 2024, Templeton et al., 2024, Gao et al., 2024] and compose those features into causal circuits [Marks et al., 2025]. We do not train SAEs, but this work is evidence that LM hidden states carry structured, manipulable features, which is the load-bearing assumption behind treating coupled hidden states as a usable communication medium.

Mixture-of-experts. MoE architectures route each token to a small subset of specialized feed-forward experts within a single transformer [Fedus et al., 2022, Jiang et al., 2024]. The Bicameral Model shares the intuition that different parts of a computation are better handled by different specialized components, but differs in three ways: the specialized components are two full, separately pretrained models rather than expert FFNs inside one; coordination is bidirectional and continuous, not a per-token routing decision; and the auxiliary model runs its own decoding loop (including tool calls) rather than a feed-forward pass on the primary's token.

Model merging and parameter composition. An alternative way to combine multiple pretrained models is to merge them at the weight level: task arithmetic [Ilharco et al., 2023], TIES-Merging [Yadav et al., 2023], DARE [Yu et al., 2024], and model soups [Wortsman et al., 2022] average or combine parameters of models that share an architecture. These methods produce a single merged model and require architectural compatibility. Our coupling keeps both models' parameters intact, trains only a small interface between their activation spaces, and works with models of different sizes or architectures provided a translation network is learned.

B Experimental details

Arithmetic (Section 4.1). The primary and auxiliary models are both Qwen2.5-0.5B-Instruct [Yang et al., 2024] (frozen, identical weights). The auxiliary model has access to a calculator tool. The interface uses PULLSTANDARD with a 3-hidden-layer translation MLP of hidden dimension 2048. Training follows a two-stage curriculum: 20 epochs of general arithmetic followed by 28 epochs of mixed arithmetic and GSM8K-type train data (Appendix C.16). Evaluation covers three benchmarks: general arithmetic (single binary expressions over $[0, 10^8]$ with log-uniform operand sampling; Appendix C.16), GSM8K [Cobbe et al., 2021] (grade-school word problems, 1,319 test examples), and GSM8K-IRL (professionally-themed transformations of GSM8K with realistic decimal quantities; Appendix C.17).

Logic puzzles (Section 4.2). The primary and auxiliary models are both Qwen3-0.6B. The auxiliary model uses ZebraDSL (Appendix C.4), a DSL we designed for this purpose, backed by the Z3 constraint solver [de Moura and Bjørner, 2008]. The interface uses PULLSTANDARD with coupling layers chosen proportionally to the best ranges found for the arithmetic problems. Training uses 10,000 procedurally generated logic puzzles per epoch, regenerated each epoch, with Z3-verified solutions. Puzzles span 2 to 6 entities and 2 to 6 attributes, covering 30 distinct clue types including direct constraints, indirect comparisons, and other patterns (Table 6). We evaluate on ZebraLogic [Lin et al., 2025], a benchmark of 1,000 logic grid puzzles ranging from 2×2 to 6×6 in size, and on GeneralZebra, a dynamically generated test suite of 1,000 puzzles matching the training distribution (Appendix C.5).

C Extended results

C.1 ZebraLogic accuracy by puzzle size

Table 3 gives the full breakdown of ZebraLogic accuracy for the bicameral maximal configuration (Qwen3-0.6B + Z3, 40 epochs) across all 25 puzzle sizes from 2×2 to 6×6.

C.2 ZebraLogic training data evolution

Table 4 shows how ZebraLogic accuracy improved across training data versions. The architecture, interface, and model were held constant; only the training data generation was changed. The v2→v4 improvement came from aligning the clue presentation format: ZebraLogic uses an ordinal framing ("There are N houses, numbered 1 to N…") while GeneralZebra originally presented entities by name, creating a distribution mismatch the model could not bridge. The v4→v5 improvement came from identifying 11 clue types present in ZebraLogic that were supported by ZebraDSL but had been overlooked in the GeneralZebra synthetic generator. These included double-indirect patterns where both sides reference entities by attribute value (e.g., "the person who drinks coffee lives next to the person who owns a cat") and mixed patterns combining one named entity with one indirect reference. These 11 types cover approximately 35% of ZebraLogic clues that were nearly absent from earlier training data (Table 6).

C.3 ZebraLogic failure mode analysis

Analyzing the 248 failures from a single bic-maximal run (75.2% accuracy, within the 5-seed distribution reported in Section 4.2), 65% are DSL value errors (the auxiliary model uses entity or attribute names not matching the declared domain), 24% are truncation (generation cut off before completing all constraints, concentrated on 5–6 entity puzzles), 9% are solver-unsat (syntactically

Table 3: ZebraLogic accuracy by puzzle size ($N \times M$) for Bicameral (0.6B + Z3). Mean $\pm$ std over 5 seeds, each evaluated on $n = 40$ puzzles per size.

N	M	Accuracy (%)	Std (%)	Group
2	2	87.0	9.3	Small
2	3	94.5	4.0	Small
2	4	92.0	6.6	Small
2	5	84.0	12.4	Small
2	6	73.0	13.2	Small
3	2	72.5	20.1	Small
3	3	93.5	3.4	Small
3	4	74.0	6.4	Medium
3	5	74.0	10.6	Medium
3	6	56.0	15.5	Medium
4	2	65.5	18.5	Small
4	3	81.5	3.4	Medium
4	4	68.0	9.1	Medium
4	5	55.0	5.7	Large
4	6	43.5	13.2	Large
5	2	72.5	11.4	Medium
5	3	66.5	8.5	Large
5	4	51.0	3.4	Large
5	5	41.0	12.0	X-Large
5	6	36.0	6.4	X-Large
6	2	67.0	10.3	Medium
6	3	62.5	7.1	Large
6	4	49.0	9.3	X-Large
6	5	35.5	7.3	X-Large
6	6	22.0	3.7	X-Large

Group	Accuracy (%)	Std (%)
Small	82.8	8.3
Medium	70.4	7.0
Large	55.7	5.3
X-Large	36.7	5.8
Overall	64.7	6.0

Table 4: ZebraLogic accuracy across training data versions. All use Qwen3-0.6B with PULLSTANDARD interface (21M params), bic-maximal configuration (40 epochs). Only the training data generation was changed between versions. v2/v4 are single runs; v5 reports mean $\pm$ std over 5 seeds. GeneralZebra is regenerated for each version using the same generator as training, so its distribution evolves across rows (19 clue types in v2, 30 in v5); ZebraLogic is fixed across all versions.

Version	ZebraLogic	GeneralZebra	Key change
v2	4.1	96.1	Baseline (19 clue types)
v4	20.5	91.3	Format alignment (10+ fixes)
v5	**64.7 $\pm$ 6.7**	**93.2 $\pm$ 0.9**	+11 double/mixed-indirect clue types

valid but logically contradictory constraints), and the remaining 2% are other errors. The dominant error, hallucinated value names, reflects a grounding problem rather than a reasoning failure: the auxiliary model correctly identifies the constraint structure but fails to map natural-language values to their declared DSL identifiers. This suggests a clear path for improvement through training data normalization.

C.4 ZebraDSL specification

ZebraDSL is a declarative constraint language designed for logic grid puzzles. Commands are semicolon-delimited and processed through a five-stage pipeline (lexer $\rightarrow$ parser $\rightarrow$ semantic analyzer $\rightarrow$ Z3 translator $\rightarrow$ query executor). The solver maintains state across commands, allowing incremental constraint formulation. Each command receives an indexed acknowledgment (`=> [N];`), and queries return solver results. Below we describe the language primitives (Table 5) and the 30 clue types used in training data generation (Table 6).

Declarations and queries. Entities are declared with `@entities:a,b,c;` and attribute domains with `@domain:color:red,blue,green;` (discrete) or `@domain:age:int[1,100];` (arithmetic). By default, all attributes are unique across entities; `@nonunique:hobby;` relaxes this. Queries take four forms: `?entity.attr;` (specific value), `?entity;` (all attributes), `?;` (satisfiability check), and `?json;` (full solution as JSON). Commands can be retracted with `!N;` (cancel command N) or `clear;` (reset all state).

Constraint syntax. Table 5 lists the constraint primitives. Operands are either *direct* (entity name, e.g., `alice.house`) or *indirect* (attribute-value reference, e.g., `@pet:dog.house`, meaning "the house number of whoever owns the dog"). All operators accept both operand forms, yielding three referencing modes: direct–direct, indirect–direct (or vice versa), and indirect–indirect.

Table 5: ZebraDSL constraint syntax. Operands L and R are either direct (`entity.attr`) or indirect (`@attr:val.attr`) references. All constraints are semicolon-terminated.

Operator	Syntax	Semantics
Equality	`L=V;` or `L=R;`	Value assignment or same-value constraint
Inequality	`L!=V;` or `L!=R;`	Value exclusion or different-value constraint
Less than	`L<R;`	Strict ordering (ordinal attributes)
Greater than	`L>R;`	Strict ordering (ordinal attributes)
Disjunction	`L=`V_1`|`V_2`|`$\cdots$`;`	Value is one of the listed options
Adjacency	`L~`n`=R;`	$\lvert L - R\rvert = n$ (bidirectional distance)
Left-of	`L+1=R;`	$R = L + 1$ (directional, e.g., immediately left)
Right-of	`L-1=R;`	$R = L - 1$ (directional, e.g., immediately right)
Entity ref.	`@attr:val=entity;`	Bind indirect reference to named entity

Clue types for training data. Training puzzles are generated procedurally with Z3-verified solutions. Each clue simultaneously produces a natural-language sentence (for the primary model's input) and one or more ZebraDSL commands (for the auxiliary model's target). Table 6 lists all 30 clue types organized by referencing mode. Clue types are sampled according to learned weights; *attribute match* and *attribute mismatch* are disabled by default as they are trivially satisfied (or impossible) under uniqueness constraints.

C.5 GeneralZebra evaluation benchmark

GeneralZebra is a dynamically generated test suite of 1,000 logic grid puzzles drawn from the same distribution as the training data. It measures in-distribution generalization and complements ZebraLogic, which tests out-of-distribution transfer.

Puzzle generation procedure. Each puzzle is constructed in three stages: (1) a random solution is created by sampling entities from one of 22 categories, sampling attributes and domain values, then randomly assigning values to entities; (2) clues are added one at a time (sampled from the 30 clue types weighted by configuration) until the constraint set yields a unique solution as verified by Z3; (3) a greedy minimization pass removes clues iteratively, restarting from the first clue after each successful removal, until no single clue can be dropped without compromising uniqueness. The result is a puzzle that is locally minimal in clue count. Table 7 lists the generation parameters.

Relation to training data. GeneralZebra puzzles are produced by the same generator code as training puzzles. Isolation relies on fresh random generation at evaluation time (1,000 puzzles) versus fresh generation at each training epoch (10,000 puzzles per epoch, 40 epochs). Given the enormous combinatorial puzzle space (22 entity categories, variable dimensions, random solutions and constraints), structural overlap is vanishingly unlikely, though train and test sets are not explicitly seed-partitioned. JSON answer validation follows the same pipeline as ZebraLogic: case-insensitive, separator-normalized comparison of every entity–attribute–value cell, with a puzzle scored correct only when all cells match.

Interpretation. The low variance across seeds on GeneralZebra ($93.2 \pm 0.9\%$) compared to ZebraLogic ($64.7 \pm 6.7\%$) reflects the in-distribution nature of the benchmark. GeneralZebra is best interpreted as a measure of whether the interface has learned to coordinate constraint formulation reli-

Table 6: All 30 ZebraDSL clue types used in training data generation. **Direct** clues reference entities by name (`e.attr`); **indirect** clues reference entities by attribute value (`@a:v.attr`); **double-indirect** clues use indirect references on both sides; **mixed** clues combine one direct and one indirect operand. Notation: e = entity name; a = attribute; v = value; t = ordinal attribute.

#	Clue type	DSL pattern	NL example
Direct (19 types)			
1	Equality	`e.a=v;`	"Alice lives in house 2"
2	Inequality	`e.a!=v;`	"Bob does not own a dog"
3	Less than	e_1`.a<`e_2`.a;`	"Alice's house is left of Bob's"
4	Greater than	e_1`.a>`e_2`.a;`	"Bob's house is right of Alice's"
5	Adjacency	e_1`.a~1=`e_2`.a;`	"Alice and Bob are neighbors"
6	Between	e_1`.a<`e_2`.a;` e_2`.a<`e_3`.a;`	"Bob is between Alice and Carol"
7	Attr. match†	e_1`.a=`e_2`.a;`	"Alice and Bob have the same hobby"
8	Attr. mismatch†	e_1`.a!=`e_2`.a;`	"Alice and Bob have different hobbies"
9	OR constraint	`e.a=`v_1`|`v_2`;`	"Alice's pet is a dog or a cat"
10	Entity ref.	`@a:v=e;`	"The Norwegian is Alice"
11	Indirect eq.	`@`a_1`:`v_1`.`a_2`=`v_2`;`	"The dog owner lives in house 3"
12	Indirect $<$	`@a:v.t<e.t;`	"The dog owner is left of Bob"
13	Indirect $>$	`@a:v.t>e.t;`	"The dog owner is right of Bob"
14	Indirect $\neq$	`@`a_1`:`v_1`.`a_2`!=`v_2`;`	"The dog owner is not Norwegian"
15	Dir. left	e_1`.t+1=`e_2`.t;`	"Alice is directly left of Bob"
16	Dir. right	e_1`.t-1=`e_2`.t;`	"Alice is directly right of Bob"
17	Indir. adj.	`@`a_1`:`v_1`.t~1=@`a_2`:`v_2`.t;`	"The Norwegian and dog owner are neighbors"
18	Distance 2	e_1`.t~2=`e_2`.t;`	"One house between Alice and Bob"
19	Distance 3	e_1`.t~3=`e_2`.t;`	"Two houses between Alice and Bob"
Double-indirect (6 types) — both operands are `@a:v` references			
20	Indir. $<$	`@`a_1`:`v_1`.t<@`a_2`:`v_2`.t;`	"The Norwegian is left of the dog owner"
21	Indir. $>$	`@`a_1`:`v_1`.t>@`a_2`:`v_2`.t;`	"The Norwegian is right of the dog owner"
22	Indir. left	`@`a_1`:`v_1`.t+1=@`a_2`:`v_2`.t;`	"The Norwegian is directly left of the dog owner"
23	Indir. right	`@`a_1`:`v_1`.t-1=@`a_2`:`v_2`.t;`	"The Norwegian is directly right of the dog owner"
24	Indir. dist. 2	`@`a_1`:`v_1`.t~2=@`a_2`:`v_2`.t;`	"One house between the Norwegian and dog owner"
25	Indir. dist. 3	`@`a_1`:`v_1`.t~3=@`a_2`:`v_2`.t;`	"Two houses between the Norwegian and dog owner"
Mixed (5 types) — one direct, one indirect operand			
26	Mixed left	`e.t+1=@a:v.t;`	"Alice is directly left of the dog owner"
27	Mixed right	`e.t-1=@a:v.t;`	"Alice is directly right of the dog owner"
28	Mixed adj.	`e.t~1=@a:v.t;`	"Alice and the dog owner are neighbors"
29	Mixed dist. 2	`e.t~2=@a:v.t;`	"One house between Alice and the dog owner"
30	Mixed dist. 3	`e.t~3=@a:v.t;`	"Two houses between Alice and the dog owner"

†Disabled by default (trivially true or impossible under uniqueness constraints).

ably, while ZebraLogic measures transfer to a fixed benchmark with different formatting conventions and entity categories.

C.6 Adapter-equivalent ablation design

The adapter-equivalent ablation isolates the contribution of the auxiliary model by retaining the interface parameters but bypassing M_a entirely. This section formalizes the signal flow.

Signal path. In the full bicameral system, reverse coupling (Eq. 2) updates the primary's hidden state at $\ell_w^{a\to p}$ using the auxiliary's hidden state at $\ell_r^{a\to p}$. In adapter-equivalent mode, M_a is never executed. Instead, the primary model's read-layer activations are passed through both translation networks in series (first $f^{p\to a}$, then $f^{a\to p}$), creating a round-trip perturbation that replaces the auxiliary-derived signal in Eq. 2:

$$\mathbf{h}_p^{(\ell_w^{a\to p})}(t) \;\leftarrow\; \big(1-\sigma^{a\to p}(t)\big)\,\mathbf{h}_p^{(\ell_w^{a\to p})}(t) \;+\; \sigma^{a\to p}(t)\,f^{a\to p}\Big(f^{p\to a}\big(\mathbf{h}_p^{(\ell_r^{p\to a})}(t)\big)\Big). \tag{5}$$

The suppression gate $\sigma^{a\to p}(t) = \mathrm{Sigmoid}\Big(g^{a\to p}(\mathbf{h}_p^{(\ell_w^{a\to p})}(t))\Big)$ is unchanged; it still reads the primary model's write-layer state. Architecturally, the round-trip $f^{a\to p}\circ f^{p\to a}$ composes the two translation MLPs into a bottleneck autoencoder ($d\to 2048\to 2048\to d\to 2048\to 2048\to d$) on

Table 7: GeneralZebra generation parameters. Unless noted, values are shared with the training data generator.

Parameter	Value
Number of puzzles	1,000
Entities per puzzle	2–6 (uniform)
Attributes per puzzle	2–6 (uniform)
Ordinal attribute ratio	0.5
Entity categories	All 22 (incl. zebralogic-style)
Clue types	All 30 (default weights)
Solution uniqueness	Z3-verified
Clue minimization	Greedy redundant-clue removal
Query type	Mixed: 40% single-attribute, 30% multiple-attribute, 30% json_table
Validation	Cell-wise match (json_table) or exact value match (single/multiple)

the primary model's own representations. The forward coupling direction ($M_p \to M_a$) is unused since there is no auxiliary model to receive it.

Training protocol. The adapter-equivalent is trained *from scratch* with the auxiliary model bypassed throughout, not evaluated post-hoc on a bicameral checkpoint. All hyperparameters, curriculum stages, data generation, and epoch counts are identical to the corresponding bicameral configuration. Only the primary model's cross-entropy loss provides gradient signal; this loss flows backward through the frozen primary layers, through $f^{a\to p}$, and through $f^{p\to a}$, so both translation networks receive gradient updates despite the auxiliary model being absent. The total trainable parameter count is identical to the full bicameral system.

Why this controls for parameter count. The adapter-equivalent tests whether the interface parameters alone, without an auxiliary model generating tool outputs, can learn a useful transformation. The interface has the same capacity (21M parameters for Qwen3-0.6B) and training signal as the bicameral system. The observed result (7.5% on ZebraLogic versus 64.7% bicameral, and 48.0% versus 96.5% on arithmetic) confirms that the gains are driven by the auxiliary model's reasoning, not by the interface acting as a parameter-efficient adapter.

C.7 Dual-target loss and masking

Section 3.1 states that the training loss is $\mathcal{L} = \mathcal{L}_p + \mathcal{L}_a$ with prompt and tool-output tokens masked. Here we detail the full masking system and loss normalization.

Loss computation. Each loss term is computed as a *masked mean*: per-token cross-entropy is multiplied element-wise by a mask tensor $\mathbf{m} \in [0,1]^T$, then summed and normalized by the mask's active mass:

$$\mathcal{L}_p = \frac{\sum_{t=1}^{T} m_t^{(p)} \cdot \ell_t^{(p)}}{\sum_{t=1}^{T} m_t^{(p)}}, \qquad \mathcal{L}_a = \frac{\sum_{t=1}^{T} m_t^{(a)} \cdot \ell_t^{(a)}}{\sum_{t=1}^{T} m_t^{(a)}}$$

where ℓ_t is the per-token cross-entropy at position t. The two terms are summed with *equal weight* (no tunable coefficient). Because each is independently normalized by its own active token count, the two loss scales are comparable even when the number of active positions differs substantially between the primary and auxiliary streams.

Masking categories. Masks are continuous-valued tensors constructed by the causality constraint solver. They are initialized to 1 (all tokens active) and then modified by six categories, applied in order:

1. **Primary prompt** ($m^{(p)} \leftarrow 0$). All input prompt tokens are masked so that only the primary model's response tokens contribute to $\mathcal{L}_p$.
2. **Aux prompt** ($m^{(a)} \leftarrow 0$). The auxiliary model's system prompt is masked, including one additional token beyond the prompt boundary. This extra token is masked because coupling activates at the first generated auxiliary token, not the last prompt token.

3. **Forced tool output** ($m^{(a)} \leftarrow 0$). Tokens injected by tool execution (e.g., calculator results `=478272;` or Z3 solver responses `=> [3];`) are masked individually. The auxiliary model did not generate these tokens, so training on them would be inappropriate and would waste interface capacity on learning to predict externally-determined outputs.
4. **Aux dropout ranges** ($m^{(a)} \leftarrow 0$). When auxiliary content blocks are stochastically dropped during data construction (controlled by an auxiliary dropout probability, default 0), the token range where the dropped content would have appeared is masked. This prevents the model from learning to predict wait tokens where content was expected.
5. **Mirrored prompt tokens** ($m^{(a)} \leftarrow 0$). When the primary model's prompt tokens are copied into the auxiliary stream (mirror configuration), these forced tokens are masked from $\mathcal{L}_a$. Note: mirror configurations were explored but are not used in any experiments reported in this paper. All reported results use a no-mirror setup where the auxiliary receives only its tool-instruction system prompt and must infer task content entirely through the hidden-state channel. Mirror configurations may be useful in practice to reduce the burden on the interface for initial value transfer.
6. **Wait-token downweighting** ($m^{(a)} \leftarrow w$, where $w \in [0, 1]$). Applied last. Non-content auxiliary tokens that survive the above categories (predominantly wait tokens between tool calls) have their mask value reduced from 1 to a configurable weight w. Content tokens (tool calls, DSL commands) retain mask value 1. This focuses $\mathcal{L}_a$ on meaningful generation rather than predicting long stretches of wait tokens. The default is $w = 1$ (no downweighting).

Categories 1–5 set mask values to 0 (binary exclusion). Category 6 is the only source of fractional mask values. The multiplication in the loss formula means that a token with $m_t = 0.01$ contributes 1% as much gradient as a token with $m_t = 1.0$.

C.8 Auxiliary dropout

During training data construction for the arithmetic domain, each auxiliary content block (e.g., a `calc(...)` call) is independently dropped with probability p. When a block is dropped, its token positions are filled with wait tokens and masked from $\mathcal{L}_a$ (category 4 above). The primary model's loss and token sequence are unchanged.

The motivation is primary-model robustness. Without dropout, if the auxiliary model fails to produce a calculator call at inference time (due to out-of-distribution inputs or gate suppression), the primary model has never encountered the absence of a reverse-coupling signal at positions where it was trained to expect one. In practice, this causes the primary model to emit space tokens indefinitely, waiting for a response that never arrives. By occasionally omitting auxiliary content during training, the primary model learns to recognize when no response is forthcoming and proceed with its best estimate.

All arithmetic experiments reported in this paper use $p = 0.004$ (approximately 1 in 250 content blocks dropped per sample). This value was selected from a sweep over $[0, 0.112]$; higher values degraded auxiliary model performance while lower values provided insufficient robustness. The ZebraLogic experiments do not use auxiliary dropout ($p = 0$).

C.9 Training data generation pipeline

The causality-tagged training data for word problems (GSM8K and GSM8K-IRL) is produced by a three-stage pipeline that transforms the target model's own outputs into corrected, annotated examples.

Stage 1: Harvesting. The target primary model (Qwen2.5-0.5B-Instruct) is run on GSM8K train (7,473 problems) and GSM8K-IRL train (7,473 problems) with temperature 0.1 and max generation length 200 tokens. On GSM8K train, the model answers 71.8% correctly; on GSM8K-IRL train, only 13.6% are correct. All outputs (correct and incorrect) are retained for annotation.

Stage 2: Minimal correction and tool annotation. Claude 3.5 Sonnet [Anthropic, 2024] processes each harvested response in a single pass. The prompt instructs Claude to:

- Identify errors in the model's reasoning and apply minimal corrections, preserving the original style, tone, and phrasing as much as possible.
- Generate `calc(expression)` entries (restricted to $+, -, \times, \div$ and parentheses) for each arithmetic step.
- Insert causality tags at their appropriate positions: `@@CALC1@@` at the earliest point where the calculation can be inferred from context, and `@@RESPONSE1@@` immediately before the numerical result first appears.

For correct responses, Claude still inserts calculator expressions and tags without modifying the text. A verification variant re-evaluates the generated `calc(...)` expressions with Python and gives Claude the exact results, allowing it to re-correct if its own arithmetic was wrong.

Stage 3: Validation. The annotated outputs are validated programmatically: `calc(...)` expressions must parse correctly and contain only digits and basic operators; tags must appear in correct order; and the final extracted answer must match the ground-truth (with tolerance for formatting differences). Records failing validation are retried (up to 3 attempts) or discarded.

Success rates. The pipeline achieves 99.1% success on GSM8K and 98.6% on GSM8K-IRL (after retries). The high success rate on GSM8K reflects that most responses are already correct and require only tag insertion. GSM8K-IRL's lower initial correctness rate (13.6%) means more corrections are needed, but Claude's single-pass correction is reliable for arithmetic word problems.

Design rationale. Using the target model's own outputs (rather than a reference model's solutions) as the starting point is deliberate: the corrected text remains in-distribution for the primary model's generation style, reducing the distribution shift between training data and inference. The minimal-correction strategy further preserves this property: Claude changes only what is mathematically wrong, leaving phrasing, step ordering, and vocabulary intact. This means the primary model trains on text it might plausibly have generated itself, with calculator results available through the coupling. This principle is important because the interface must learn to steer the primary model's own generation patterns rather than imitate a different model's style. A more principled variant of this pipeline, in which Claude makes atomic single-error fixes and the target model regenerates all subsequent tokens (ensuring $>95\%$ of training tokens are the target model's own output), is under development for future work.

C.10 Causality-constrained data alignment

Section 3.2 describes the causality constraint system at a high level. Here we detail the data structures, constraint solving algorithm, and tag anchoring patterns.

Problem statement. Both models must produce token sequences of equal length for lockstep alignment. The auxiliary model's content (tool calls and their results) must be placed so that: (a) the auxiliary model does not act on information the primary model has not yet produced at that token position (causal ordering), and (b) tool results are available before the primary model needs them (timing). Between active tool calls, the auxiliary stream is filled with wait tokens.

Tag anchoring. Each training example embeds *causality tags* (markers of the form `@@TAG_NAME@@`) directly into the primary model's response text. Each tag marks a logical boundary, such as the end of a question or the start of an answer. An auxiliary content block specifies two tags: an `after_tag` (the block must not start until the primary model has passed this position) and a `before_tag` (the block must complete before the primary model reaches this position). The tag processor resolves each tag to a token index by: (1) recording the tag's character position in the tagged text, (2) stripping all tags to produce clean text, (3) tokenizing the prefix up to the adjusted character position and counting tokens. The clean text (without tags) is what both models actually see during training and inference.

Constraint solver. Given a list of auxiliary content blocks with their tag-derived placement windows, the solver walks the blocks sequentially, maintaining a cursor position. For each block, it:

1. Tokenizes the auxiliary content and any forced tool output to compute token lengths.

2. Computes the valid placement window: $[\max(\textit{after_pos}, \textit{cursor}),\ \textit{before_pos} - \textit{content_len}]$.
3. Selects a position within this window using the scheduling strategy (see below).
4. Checks for constraint violations: a *before-violation* if the block's end exceeds *before_pos* (tool result arrives too late), or an *after-violation* if the block's start precedes *after_pos* (causal ordering broken).
5. If violated, applies a fallback policy: `allow` (keep despite violation), `drop_ar_output` (skip this block), `drop_sample` (discard the entire example), or `primary_wait` (insert space tokens into the primary sequence to create room; see below).
6. Advances the cursor past the placed content and forced output.

Scheduling strategies. Four strategies determine where within a valid window the auxiliary content is placed:

- **Eager**: earliest valid position, $\max(\textit{after_pos}, \textit{cursor})$. The auxiliary model fires its tool call as soon as it causally can.
- **Lazy**: latest valid position, *before_pos* − *content_len*, clamped to the window floor. The auxiliary model waits until the last possible moment.
- **Random**: uniform sample within the valid window. This is used in all reported experiments.
- **Balanced**: identical to random in the current implementation.

Random placement provides data augmentation: the same logical example produces different token-level alignments across epochs (combined with per-epoch regeneration of the underlying problems).

Primary-wait fallback. When a before-violation occurs (the auxiliary content does not fit before the `before_tag` position), the `primary_wait` policy *modifies the primary token sequence* to create room. It calculates the number of extra tokens needed (*required_end* − *before_pos*) and inserts that many space tokens into the primary stream at the constraint boundary. All subsequent positions shift accordingly. The effect: the primary model emits extra space tokens to "wait" for the auxiliary model to finish its tool call. This is the only policy that alters the primary model's training data. In our experiments, before-violations use `primary_wait`; after-violations (causal ordering) use `drop_sample`.

Worked example: arithmetic. Consider the problem "What is 564×848?" with answer 478,272. Tags are placed in the primary response as follows:

Primary (tagged): `564 * 848 equals @@ANSWER_READY@@478272.`

Input (tagged): `What is 564 @@QUESTION_END@@* 848?`

Aux content: `calc(564*848)` with forced output `=478272;`
`after_tag`: QUESTION_END `before_tag`: ANSWER_READY

The constraint solver resolves QUESTION_END to the token index of "564" and ANSWER_READY to the token index of "478272". The auxiliary block `calc(564*848)` must be placed after the primary has produced the first operand (so the auxiliary model has "seen" it via hidden-state coupling) and before the primary needs the answer. Between these positions, the auxiliary stream contains wait tokens; at the selected position, the auxiliary stream contains `calc(564*848)=478272;` (content + forced output, the latter masked from loss).

Worked example: logic puzzle. For zebra puzzles, the think block uses a chain of tags that anchor each DSL command to its corresponding natural-language restatement:

Primary think block (abridged)

```
@@ENT@@Entities:  Alice, Bob, Carol
@@A1@@- Pet:  dog, cat, fish
@@A2@@- House:  1, 2, 3
@@C1@@Alice lives in house 2
@@C2@@Bob does not own a dog
@@SQ@@...solver informs me...
@@SR@@{solution JSON}
```

Aux stream

```
@entities:alice,bob,carol; => [1];
@domain:pet:dog,cat,fish; => [2];
@domain:house:1,2,3; => [3];
alice.house=2; => [4];
bob.pet!=dog; => [5];
?json; => {...};
```

Each auxiliary command's placement window is bounded by adjacent tags: the entity declaration is placed between ENT and A1, the first domain between A1 and A2, the first constraint between C1 and C2, and the `?json;` query between SQ and SR. This chaining ensures monotonic ordering: every DSL command appears in the auxiliary stream at a position where the primary model has just restated the corresponding piece of the puzzle, and before it moves on to the next. Gray text shows forced tool output (Z3 acknowledgments and results), which is masked from the auxiliary loss.

C.11 Reinforcement learning on the interface

After supervised fine-tuning, the interface parameters can be further optimized with reinforcement learning. We implement GRPO [Shao et al., 2024] with the following design:

- **What is optimized.** Only the neural interface parameters θ_ϕ (including the suppression gate). Both LLMs remain frozen and in eval mode.
- **Trajectory generation.** For each prompt, K trajectories (default 96) are generated through the full coupled pipeline with temperature sampling, producing diverse primary–auxiliary token sequences.
- **Reward.** Outcome-based: binary 1/0 for answer correctness. An optional tool-use reward component scores whether the auxiliary model invoked tools appropriately, though empirically this has minimal effect beyond the outcome signal.
- **Advantage computation.** Per-prompt group normalization: rewards are mean-centered and std-normalized within each prompt's trajectory group. A subset (default 16) is selected for training, balancing high-reward and low-reward trajectories.
- **Policy gradient.** PPO-style clipped objective ($\epsilon = 0.2$) over the token distributions of both models, with DAPO-style loss normalization. Gradients flow through both models' forward passes but only update the interface.

Preliminary results. On the arithmetic domain (starting from an SFT-trained PULLSTANDARD checkpoint), RL training yields consistent improvements on held-out word problems: GSM8K-IRL accuracy rises from 16.1% (SFT) to 20.8% (+4.7pp), with no regression on calculator-based arithmetic (96–97%) or GSM8K (39–43%). Results are reproducible: four identical RL runs show standard deviations below 1% on all benchmarks. We swept learning rate (1e-6 to 3e-3), sampling temperature, reward weighting, and trajectory selection strategy across 150+ runs; the optimal learning rate (3e-6) is an order of magnitude below SFT, and performance is robust to other hyperparameter choices.

RL as a timing optimizer. SFT requires carefully engineered causality constraints (Appendix C.10) to align the two token streams, and the quality of this alignment affects performance. RL sidesteps this: because it optimizes the interface end-to-end on outcome reward, it can learn when the suppression gate should fire and how strongly, effectively discovering coordination timing from the reward signal rather than from hand-crafted placement windows. We leave a full investigation of RL training, including application to the ZebraLogic domain and scaling to larger models, to future work.

C.12 Python tool coupling examples

Using twin Qwen3-4B models with an identity interface (trained on NuminaMath-TIR [AI-MO/NuminaMath-TIR contributors, 2024]), the auxiliary model receives no problem text, only hidden-state signals from the primary model at intermediate layers. Despite this, it generates problem-specific Python code.

Training data construction. The 72k NuminaMath-TIR examples are split into two streams without correctness filtering. Each example's serial solution (interleaved reasoning text and `python`/`output`

blocks) is parsed into segments. The primary stream receives all reasoning text wrapped in a think block, with causality tags inserted where code blocks originally appeared. The auxiliary stream receives the extracted code blocks, with execution outputs attached as forced tokens (masked from loss). Causality constraints anchor each code block between tags so it is placed after the primary has produced the relevant reasoning context and before the primary needs the result.

Results. Aggregate accuracy (62.5% on MATH) does not exceed the unaugmented Qwen3-4B with thinking enabled (81.6% on MATH at 32k tokens). However, on 127 MATH problems (2.5% of the benchmark), the bicameral system produces correct answers where thinking alone fails. Below are examples illustrating what the hidden-state channel transmits; full coupling traces are shown in Figures 10–12.

Example 1: Modular arithmetic (wins vs. thinking baseline). Problem: "Tim is doing a cakewalk with 11 steps. He takes his first step on step 1 and takes a total of 139 steps, walking in a circle. Which step does he end on?" The unaugmented Qwen3-4B with 32k-token thinking gets this wrong.

```
final_step = (1 + 139 - 1) % 11
print(final_step)  # 7
```

Three lines. The auxiliary recovered both key numbers (11, 139) and the correct offset formula from hidden states alone. Answer: correct.

Example 2: Implicit algebraic simplification. Problem: "Let $n = 2^4 \cdot 3^5 \cdot 4^6 \cdot 6^7$. How many natural-number factors does n have?"

```
exponent_2 = 23
exponent_3 = 12
num_factors = (exponent_2 + 1) * (exponent_3 + 1)
print(num_factors)  # 312
```

The auxiliary emitted the consolidated prime exponents (23, 12) rather than the raw values (4, 5, 6, 7), performing the simplification $4^6 = 2^{12}$, $6^7 = 2^7 \cdot 3^7$ implicitly. Answer: correct.

Example 3: Base conversion (wins vs. thinking baseline). Problem: "Captain Rusczyk tracked down a pirate who had stolen 2345_6 dollars. After winning a duel, he demands 41324_5 dollars. How much has the pirate gone in debt? Express in base 10." The unaugmented model gets this wrong.

```
base_6 = 2*(6**3) + 3*(6**2) + 4*(6**1) + 5*(6**0)
base_5 = 4*(5**4) + 1*(5**3) + 3*(5**2) + 2*(5**1) + 4*(5**0)
print(base_6 - base_5)  # -2145
```

The auxiliary reconstructed both digit strings (2345 in base 6, 41324 in base 5) and both bases from hidden states. Answer: correct (debt = 2145).

Example 4: Recursive sequence with 7 recovered parameters. Problem: "Given x_1=211, x_2=375, x_3=420, x_4=523 and $x_n = x_{n-1} - x_{n-2} + x_{n-3} - x_{n-4}$, find $x_{531} + x_{753} + x_{975}$."

```
x = [0] * 1000
x[1]=211; x[2]=375; x[3]=420; x[4]=523
for n in range(5, 976):
    x[n] = x[n-1] - x[n-2] + x[n-3] - x[n-4]
print(x[531] + x[753] + x[975])  # 898
```

All four initial values, the alternating-sign recurrence, and the three target indices were recovered from hidden states. Answer: correct.

C.13 Layer-wiring constraints and inference

Acyclicity constraint. The coupling configuration is defined by four layer indices: the primary read layer ℓ_r^p, primary write layer ℓ_w^p, auxiliary read layer ℓ_r^a, and auxiliary write layer ℓ_w^a. Since activations flow upward through layers (from layer 0 to layer L), cross-model coupling must not introduce a cycle in the activation dependency graph. The four coupling operations must admit a topological sort that respects layer ordering within each model.

The coupling order is determined by the primary model's layer relationship. When $\ell_r^p \leq \ell_w^p$ (**standard order**), the primary is read first (clean, before receiving any perturbation), its signal is written to the

auxiliary at ℓ_w^a, the auxiliary continues to ℓ_r^a and is read, and that signal is written back to the primary at ℓ_w^p. When $\ell_w^p < \ell_r^p$ (**reversed order**), the auxiliary is read first (clean), written to the primary, the primary continues to its read layer, and that signal is written back to the auxiliary.

Within each model, read and write layers can independently take any relative ordering (read $<$ write, read $=$ write, or write $<$ read), giving 9 combinations. However, one combination is invalid: when the primary uses reversed order ($\ell_w^p < \ell_r^p$), the auxiliary must have $\ell_r^a < \ell_w^a$ (strict). Otherwise, the primary-to-auxiliary perturbation is injected at a layer that is never used for the auxiliary's final forward pass, causing it to be silently lost. All other 8 combinations are valid. When read $=$ write on a given model, the operation order follows the coupling direction: the model that goes first reads clean then receives its write; the model that goes second receives its write then is read (picking up the perturbed state).

The 890-configuration sweep samples from this space of valid configurations.

KV cache with perturbed activations. During inference, each model maintains a KV cache for efficient autoregressive decoding. When the interface injects a perturbation at a write layer, the perturbed hidden state continues through the remaining layers and is stored in the KV cache at each subsequent layer. This means future tokens attend to the perturbed representations when computing attention over past positions. The coupling effect thus persists across time steps through the cache, even though the interface only operates on the current token's hidden states. Position IDs are derived from the cache sequence length to ensure correct positional encoding during incremental decoding.

C.14 Ablations and scaling

All experiments use AdamW with learning rate 4×10^{-4}, gradient clipping at 1.0, and batch sizes of 2–8 (with gradient accumulation where noted). The learning rate was not tuned per experiment. Table 8 lists the interface configurations evaluated.

Table 8: Interface configurations. All use ReLU activations and identical twin models (same weights for primary and auxiliary). Adapters use α=128 where present.

Domain	Model (d)	Interface	Params	Translation	Suppression	Adapters
Arithmetic	Qwen2.5-0.5B (896)	PULLSTD	16M	3L, h=2048	3L, h=64	None
Arithmetic	Qwen2.5-0.5B (896)	SCALARID	803K	Identity	2L, h=64	r64, L7/10/15
Arithmetic	Qwen2.5-0.5B (896)	ELEMID	919K	Identity	2L, h=64	r64, L7/10/15
ZebraLogic	Qwen3-0.6B (1536)	PULLSTD	21M	3L, h=2048	3L, h=64	None
Python tool	Qwen3-4B (2560)	SCALARID	2.3M	Identity	2L, h=64	r64, L11/15/23

Coupling depth. We swept five same-depth coupling configurations (layers 6/18, 8/16, 10/15, 10/20, 12/14) across both identity architectures with and without adapters (40 configurations total). All mean accuracies in this section are averaged across general arithmetic, GSM8K, and GSM8K-IRL test sets. Mid-layer coupling at depths 10–15 and 10–20 is consistently optimal, achieving 54.9% and 51.9% mean accuracy respectively. Shallow coupling (6/18, 8/16) degrades all metrics, falling to 25–31%. This pattern is consistent across interface types.

Adapter placement. With identity interfaces, adapter placement relative to the coupling layers is the dominant factor. Across 32 configurations testing eight placement strategies at two coupling depths, adapters at the coupling layers produce the best combined results (46.7% mean across all three benchmarks), while early-layer adapters (layers 3/5/7) are worse than no adapters at all (22.0% versus 34.5%). Adapter rank 32–64 is the sweet spot; the suppression network size has little effect.

Curriculum effect. The arithmetic warm-up phase is essential for word-problem transfer on harder problems. Training 48 epochs on GSM8K + GSM8K-IRL alone (no arithmetic stage) achieves comparable GSM8K accuracy (41.7% vs 39.9% with curriculum) but nearly halves GSM8K-IRL performance (11.8% vs 21.6%). Without arithmetic practice, the auxiliary model still learns to trigger `calc()` expressions (78–100% trigger rate, 99% syntactically valid), but the primary model fails to reliably integrate the calculator results into its reasoning through the coupling. The pure arithmetic stage teaches the interface to faithfully transfer quantities between the two models' hidden states, while the word-problem examples teach the system *when* to invoke the calculator. GSM8K-IRL's realistic decimal quantities amplify this effect: its sub-computations require precise multi-step decimal arithmetic where the primary model struggles to succeed without reliable calculator integration.

Robustness to layer configuration. In a separate sweep of 890 layer wiring configurations using PULLSTANDARD, 95% of configurations improved over the primary-only arithmetic baseline and 23.7% improved on GSM8K-IRL, confirming that the coupling mechanism is robust to exact layer placement within the mid-to-late optimal region.

Model scaling. Preliminary experiments confirm the architecture functions at larger scales, including asymmetric pairings (32B primary with 0.6B auxiliary using pipeline parallelism). However, configurations were not optimized at larger scales, and word-problem degradation persisted, likely due to our initial experiments reusing annotated training data initially harvested from the Qwen2.5-0.5B model. Characterizing scaling behavior and emergent properties of the bicameral configuration is a direction for future work.

C.15 Layer-wiring sweep

The 890-configuration sweep varies which transformer layers are tapped for coupling in both models across all valid coupling-order regimes. Figure 6 shows the best accuracy achieved by any configuration using each layer value (marginalized over all other layer choices). Error bars show Bernoulli standard error ($\sqrt{p(1-p)/n}$) from the finite test set, reflecting measurement uncertainty.

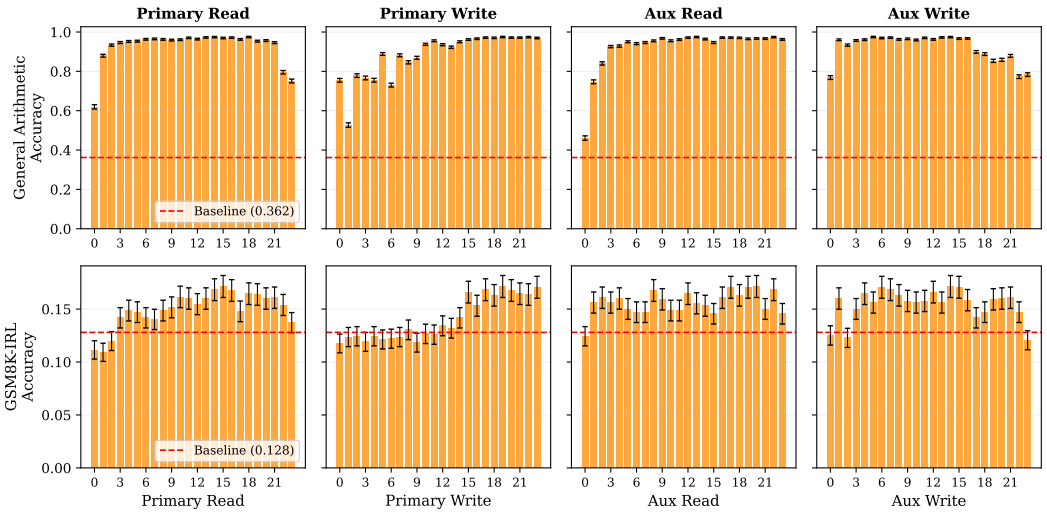


Figure 6: Best performance for each layer value across 890 configurations, marginalized over other layer choices. **Top:** General arithmetic. **Bottom:** GSM8K-IRL. Red dashed line: primary-only baseline. Mid-to-late layers (10–20) consistently achieve the highest ceilings. Error bars: Bernoulli SEM.

Statistical significance methodology. To determine whether a configuration significantly outperforms the primary-only baseline, we use a two-sample test based on the Bernoulli standard error. For a configuration with accuracy p_c on n_c samples and a baseline with accuracy p_b (averaged over 8 runs, SEM s_b): the combined uncertainty is $s = \sqrt{s_b^2 + p_c(1-p_c)/n_c}$, and a configuration is classified as significantly better if $(p_c - p_b) > 2s$ (approximately 95% confidence). Of the 890 configurations, 95.4% improve over the primary-only baseline on general arithmetic ($n = 2{,}000$), of which 94.7% (843 configs) are statistically significant. On GSM8K-IRL ($n = 1{,}319$), 23.7% improve over the baseline, of which 6.6% (59 configs) reach significance. On GSM8K ($n = 1{,}319$), 0% improve.

Figure 7 shows the best improvement over the primary-only baseline for each projected layer pair, revealing which layer regions contain at least one configuration that substantially outperforms the baseline.

Interpreting the heatmap structure. The heatmaps in Figure 7 reveal clear triangular structure that aligns with the architecture's causal constraints. For primary model layers, the high-performance region forms an upper-left triangle corresponding to primary read $<$ primary write (standard ordering, where the primary model is read first). For auxiliary model layers, the high-performance region forms a lower-right triangle corresponding to auxiliary write $<$ auxiliary read. Together, these define the

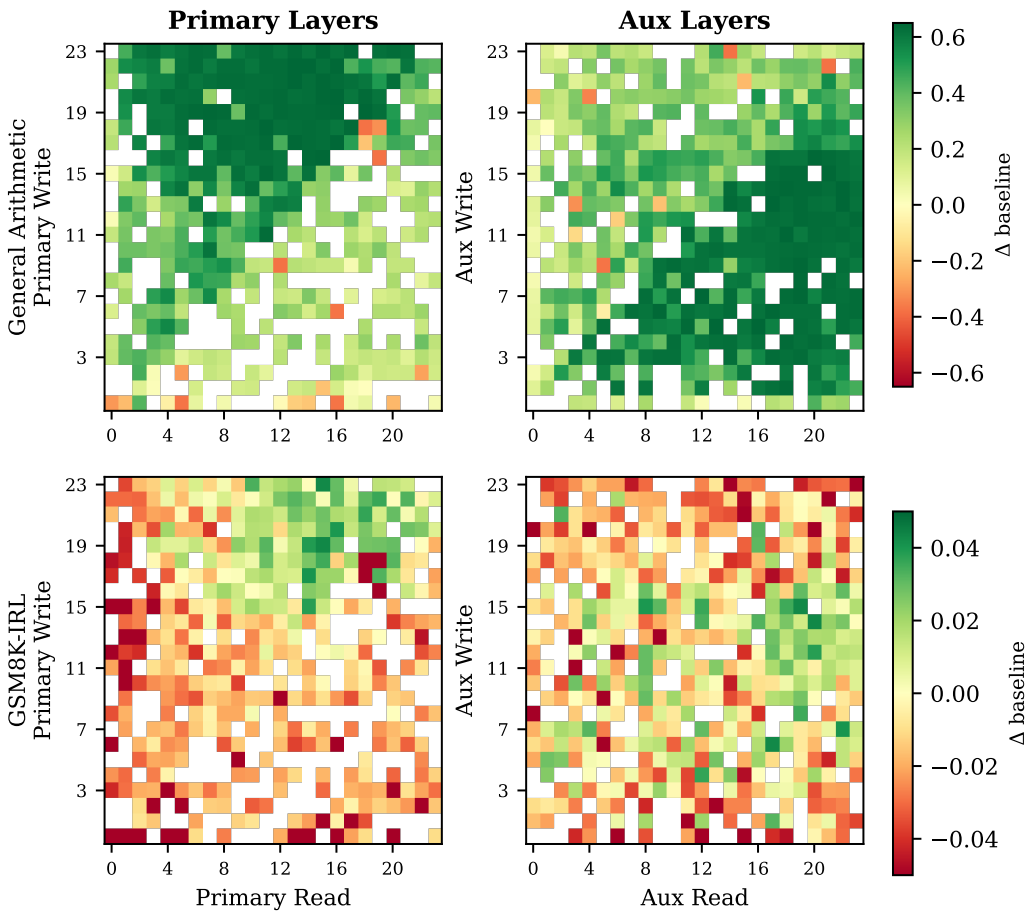


Figure 7: Best improvement over primary-only baseline for each projected layer pair across 890 configurations. **Top:** General arithmetic. **Bottom:** GSM8K-IRL. Green regions contain configurations that substantially beat the baseline; the optimal region (mid-to-late layers for both read and write) is clearly visible. White cells have no data for that layer pair.

"standard ordering with round-trip within one token" regime: the primary is read, its signal is written to the auxiliary at an earlier layer, the auxiliary processes this signal through its intermediate layers, and its response is read at a later layer to be written back to the primary, all within a single generation step.

This is precisely the configuration depicted in Figure 1 and the regime we initially hypothesized would be optimal before implementing the sweep. Configurations outside this triangle still work (the acyclicity constraint is satisfied), but the round-trip is broken: if the auxiliary is read before being written to (auxiliary read $<$ auxiliary write), both couplings still fire within the same token, but the reverse coupling cannot be informed by the latest forward coupling. The auxiliary's response at its read layer reflects only prior tokens' forward signals, not the current one. The sweep confirms that this broken round-trip is harmful but not fatal.

The heatmaps also reveal a clear asymmetry between standard and reversed coupling order. Reversed order (primary write $<$ primary read, where the auxiliary is read first) performs consistently worse, even within the round-trip regime. The likely explanation is that reversed order has the auxiliary model "drive" the interaction: its hidden states influence the primary model before the primary's own representations can inform the auxiliary about the current task context. Since the primary model sees the actual problem and the auxiliary does not, this ordering is counterproductive. In this task setup, the auxiliary likely benefits more from responding to the primary's signal than from leading the interaction.

Within the optimal triangular region, performance concentrates further in a specific subregion: mid-to-late primary read (layers 10–15) paired with a primary write a few layers beyond, and mid-to-late auxiliary write paired with a later auxiliary read. This pattern is consistent across both general arithmetic and GSM8K-IRL, suggesting that the optimal coupling taps representations after the models have built sufficient contextual understanding (past the early layers) but before the final layers commit to specific token predictions.

C.16 General arithmetic benchmark

The general arithmetic evaluation set is generated synthetically with the same procedure used for training data, ensuring no overlap between training and evaluation (different random seeds, regenerated each epoch during training). Each problem is a single binary expression $a \circ b$ where $\circ \in \{+, -, \times, \div\}$.

Number sampling. Operands are drawn from $[0, 10^8]$ using a mixed sampling strategy: with probability 0.05 the value is sampled uniformly, otherwise log-uniformly ($10^{U(0,8)}$). Log-uniform sampling ensures coverage across magnitudes (from single-digit to hundred-million-scale) while the small uniform component prevents complete absence of very large values. Each operand is then rounded to a randomly selected precision level (a power of 10 from 10^{-2} to 10^3, sampled uniformly). The precision controls decimal granularity, not magnitude: a large sample like 45,283,899.78 becomes 45,284,000 at precision 10^3, 45,283,900 at precision 10^1, or remains 45,283,899.78 at precision 10^{-2}. The minimum representable value is clamped to the precision factor. Operands are negated with probability 0.01.

Operator handling. All four operators are equally likely. Division avoids zero denominators; subtraction swaps operands to produce positive results. Answers are computed by the calculator tool and verified to arbitrary precision ($\leq$5 decimal places in the formatted output).

Template formatting. Problems are presented in natural language using one of seven template categories (basic, conversational, LaTeX, formal, casual, mathematical, verbose), with 90% basic templates (e.g., "Calculate 423 + 15,000") and the remainder distributed across other styles. This variety teaches the primary model to recognize arithmetic requests in diverse phrasings.

Evaluation protocol. Each evaluation set consists of 10,000 problems generated with a fixed seed (42). The model receives the problem text and must produce the correct numerical answer. We report exact-match accuracy after normalizing formatting (commas, trailing zeros).

Training curriculum. The 48-epoch curriculum uses two stages: (1) epochs 0–20: 20,000 pure arithmetic problems regenerated each epoch; (2) epochs 20–48: a mixed dataset of $\sim$15,800 samples (46.7% GSM8K-IRL, 46.9% GSM8K, 6.3% general arithmetic), with the arithmetic portion regenerated each epoch. The transition to mixed data teaches the interface to route word problems to the calculator while preserving the primary model's chain-of-thought reasoning.

C.17 GSM8K-IRL benchmark

GSM8K-IRL ("In Real Life") is a synthetically generated benchmark that transforms GSM8K problems into harder, professionally-themed variants with realistic non-round quantities. We generate it using Claude 3.5 Sonnet [Anthropic, 2024] via a structured transformation pipeline.

Generation. Each of the 7,473 GSM8K training problems (and separately, the 1,319 test problems) is sent to Claude 3.5 Sonnet with a transformation prompt instructing it to:

- Replace child/school contexts with adult professional scenarios (workplace, finance, engineering, science, legal, etc.)
- Replace simple round numbers with realistic decimal quantities (e.g., \$20 $\rightarrow$ \$19.89)
- Maintain the same number of calculation steps and the same arithmetic operators ($+, -, \times, \div$ only)
- Produce a step-by-step solution and a final extracted numerical answer

Verification. A separate verification pass ensures solution correctness. Claude extracts all arithmetic sub-expressions from the generated solution, Python `eval()` computes each expression exactly, and Claude reviews the full solution given the exact calculation results to determine whether the final answer is correct. Problems with incorrect solutions are flagged.

Error rate. The realistic decimal quantities introduce systematic rounding and truncation errors at a $\sim$2% rate (155 suspect ground-truth labels out of 7,473 training items), approximately 14$\times$ higher than GSM8K's $\sim$0.15% error rate. These errors arise because multi-step decimal arithmetic is genuinely harder, and the generation model occasionally rounds intermediate results. We do not filter these from the training set, as the 2% noise rate has negligible impact on training signal.

Dataset statistics. The train split contains 7,473 problems (one per GSM8K train example) and the test split contains 1,319 problems (one per GSM8K test example). Problems span diverse professional domains including finance, engineering, chemistry, logistics, and project management.

Evaluation. We evaluate by extracting the numerical answer from model output (last `\boxed{}` match, or last number in the final sentence) and comparing against the ground truth using `math.isclose` with relative tolerance 10^{-5} and absolute tolerance 10^{-8}. This is effectively exact match for integer answers but accommodates minor floating-point discrepancies on decimal results. The test split is a transfer benchmark: training uses the IRL train split as part of the mixed-data curriculum (stage 2), while evaluation uses the held-out IRL test split derived from different seed problems.

C.18 Compute resources

Table 9 summarizes GPU-hours across all experiments reported in this paper. The total is approximately 26,000 GPU-hours, predominantly on NVIDIA L40S (48GB) and H200 (80GB) GPUs.

Table 9: Compute resources by experiment category.

Category	Runs	GPUs/run	Hours/run	GPU-hours
Arithmetic layer sweep (890 configs)	890	1 L40S	$\sim$17	15,130
Arithmetic identity/adapter sweeps	230	1 L40S	7–17	2,760
Arithmetic multiseed (2$\times$5 seeds)	10	1 L40S	$\sim$17	170
ZebraLogic training (v2–v5, L40S)	30	1 L40S	4–12	180
ZebraLogic v5 multiseed (5 seeds)	5	1 H200	$\sim$55	275
ZebraLogic v5 extended (80–160 ep)	5	1 H200	$\sim$110	550
ZebraLogic adapter-equivalent	2	1 H200	$\sim$55	110
Python tool / TIR (Qwen3-4B)	27	8 L40S/H200	$\sim$11	2,376
Communication onset (train + eval)	106	1 L40S	1–8	320
RL experiments	150	1 L40S	$\sim$6	900
Multi-GPU scaling (8B, 32B)	4	8 L40S	$\sim$80	2,560
Evaluation (all benchmarks)	200+	1–8	1–2	500
Total				**$\sim$26,000**

Training times per run ranged from 4 hours (20-epoch ZebraLogic on H200) to 18 hours (48-epoch arithmetic curriculum on L40S). The 890-experiment layer sweep dominates total compute (58%). All sub-1B experiments used single GPUs; the 4B Python-tool and 8B/32B scaling experiments used 8-GPU pipeline parallelism.

C.19 Full coupling visualizations

The following figures show complete token-by-token coupling traces for successful generations across all three tool domains. Each visualization displays the full primary and auxiliary token streams with coupling strength at every position, revealing the temporal coordination patterns the interface learns from task loss alone. In all figures, **blue** indicates forward coupling ($M_p \to M_a$), **red** indicates reverse coupling ($M_a \to M_p$), and **green** highlights forced output tokens where tool results are injected into the auxiliary's hidden-state stream.

C.19.1 Arithmetic with calculator

Figure 8 shows the complete generation for a multi-step word problem requiring five arithmetic operations and a conditional discount.

C.19.2 Logic puzzle with Z3 solver

Figure 9 shows the complete generation for a 3×3 ZebraLogic puzzle with 4 attributes and 6 clues.

C.19.3 Python tool

Figures 10–12 show complete traces for the Python-tool configuration (twin Qwen3-4B), where the auxiliary model receives *no problem text*, only hidden-state signals at intermediate layers. Despite this, it generates problem-specific Python code.

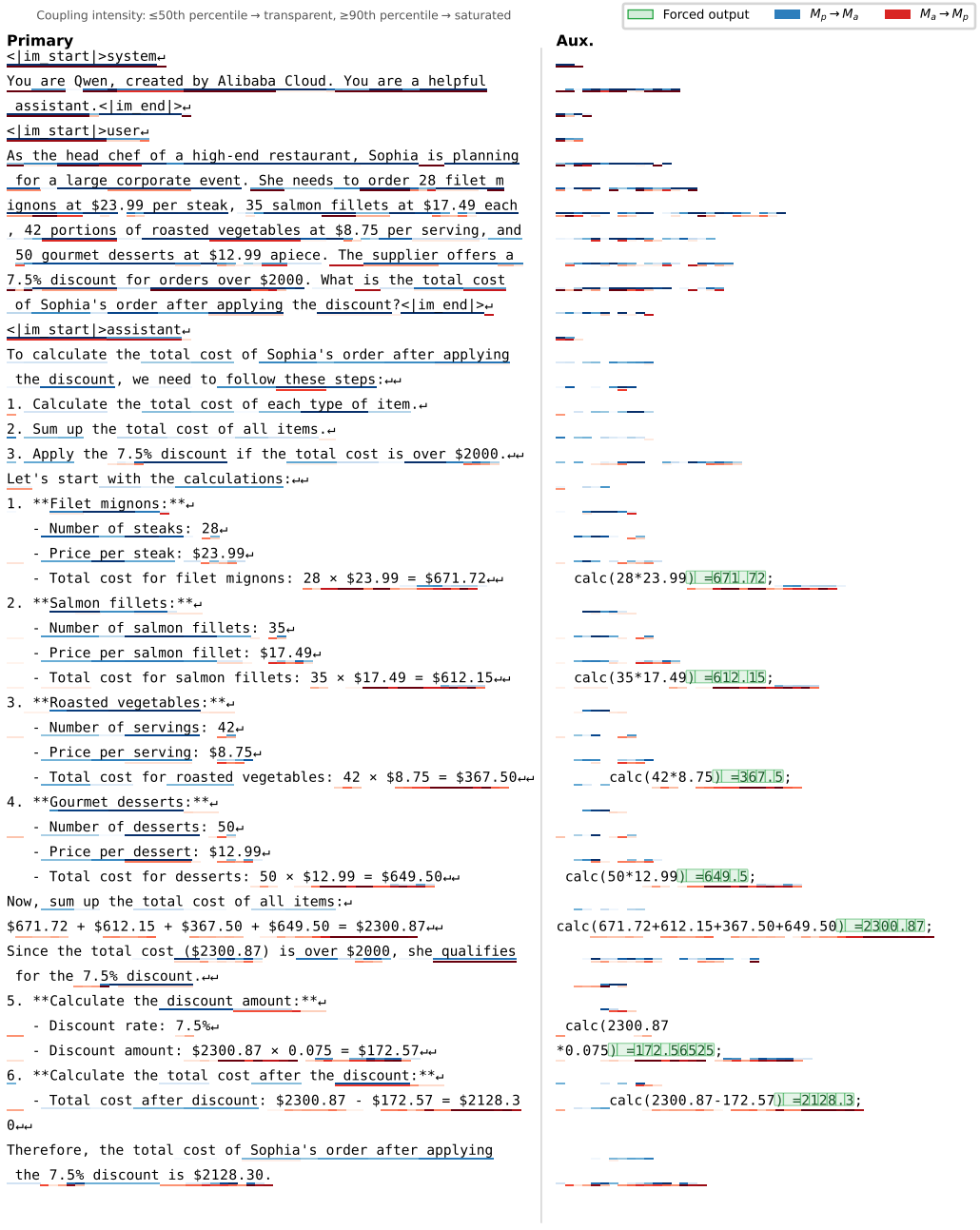
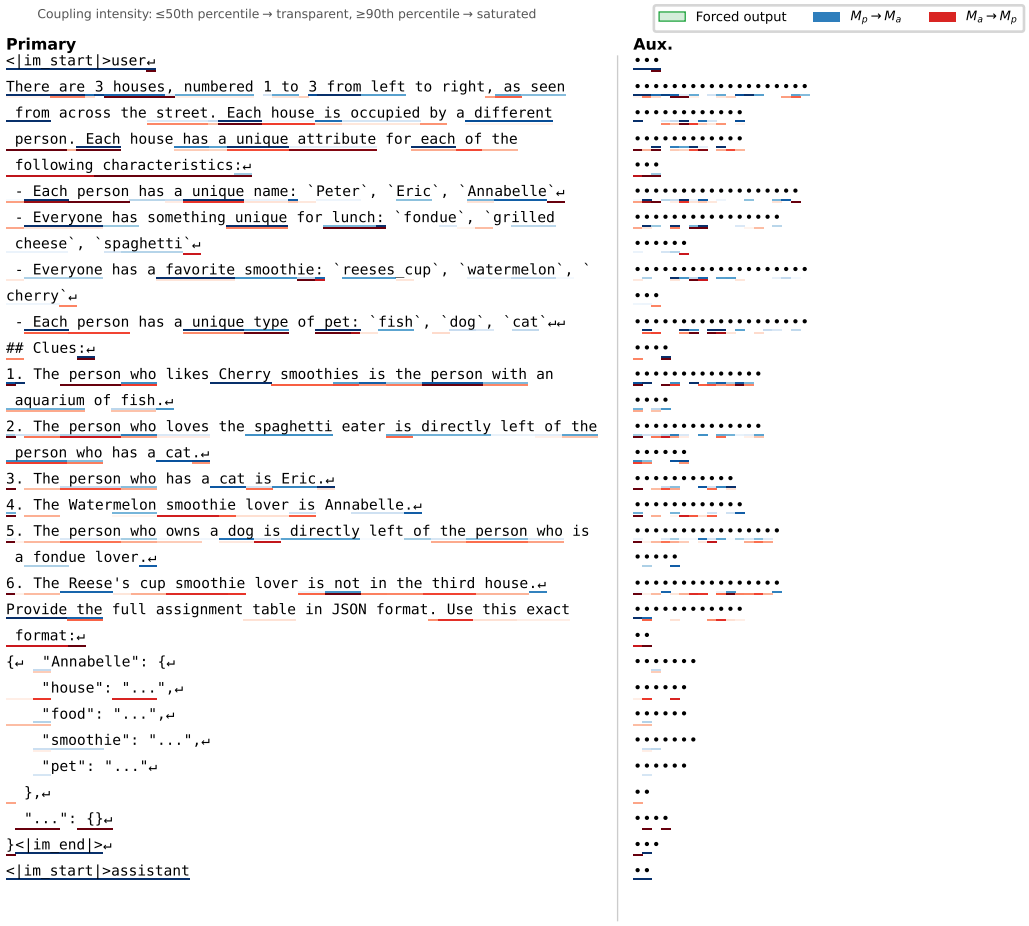


Figure 8: **Full coupling trace: multi-step arithmetic word problem** (Qwen2.5-0.5B-Instruct, scalar identity interface, calculator tool). The auxiliary (right column) waits with spaces until the primary emits relevant tokens, then issues sequential `calc(...)` calls. Forward coupling (blue) is sustained while the primary emits dollar amounts and quantity words; reverse coupling (red) is near-zero except at forced output positions (green, `=671.72`, `=612.15`, etc.) where calculator results flow back, or are recalled when the primary model references those values later (e.g., for the final answer). The primary model incorporates each result into its chain-of-thought with single-token latency. Figure 5 in the main text shows a zoomed excerpt of this trace (tokens 464–515).


Coupling intensity: ≤50th percentile → transparent, ≥90th percentile → saturated
Forced output
$M_p \to M_a$
$M_a \to M_p$
Primary
Aux.
<|im start|>user↵
There are 3 houses, numbered 1 to 3 from left to right, as seen from across the street. Each house is occupied by a different person. Each house has a unique attribute for each of the following characteristics:↵
- Each person has a unique name: `Peter`, `Eric`, `Annabelle`↵
- Everyone has something unique for lunch: `fondue`, `grilled cheese`, `spaghetti`↵
- Everyone has a favorite smoothie: `reeses cup`, `watermelon`, `cherry`↵
- Each person has a unique type of pet: `fish`, `dog`, `cat`↵↵
Clues:↵
1. The person who likes Cherry smoothies is the person with an aquarium of fish.↵
2. The person who loves the spaghetti eater is directly left of the person who has a cat.↵
3. The person who has a cat is Eric.↵
4. The Watermelon smoothie lover is Annabelle.↵
5. The person who owns a dog is directly left of the person who is a fondue lover.↵
6. The Reese's cup smoothie lover is not in the third house.↵
Provide the full assignment table in JSON format. Use this exact format:↵
{↵ "Annabelle": {↵
"house": "...",↵
"food": "...",↵
"smoothie": "...",↵
"pet": "..."↵
},↵
"...": {}↵
}<|im end|>↵
<|im start|>assistant

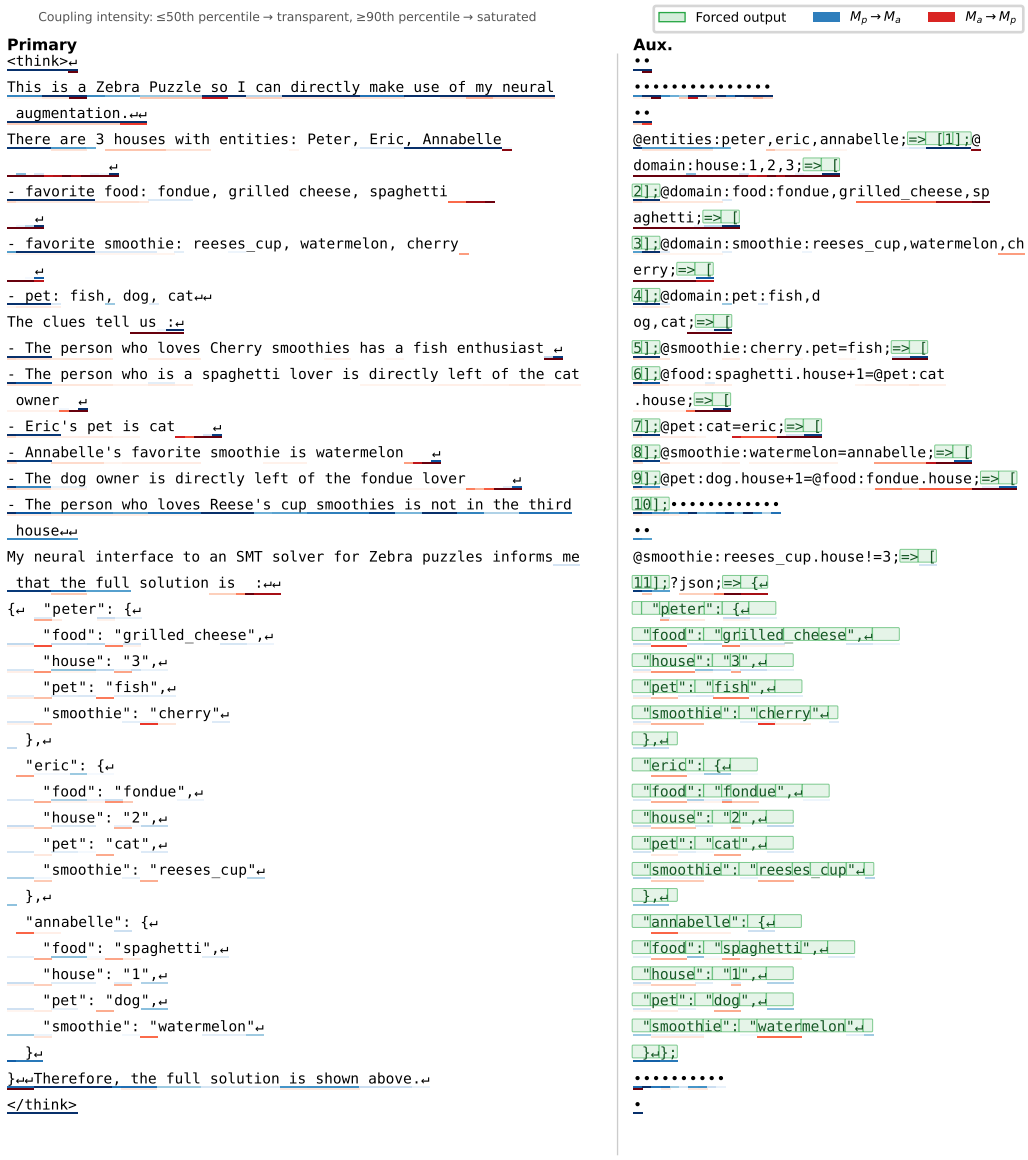


Figure 9: **Full coupling trace: 3×3 logic puzzle** (Qwen3-0.6B, standard interface, ZebraDSL/Z3 tool). This puzzle is modified from a ZebraLogic benchmark instance: "Arnold" → "Annabelle," "pizza" → "fondue," and "desert" → "Reese's cup." None of these replacement values appear anywhere in the training data across any entity category, testing out-of-distribution generalization. The auxiliary (right column) emits wait tokens (•) during Phase 2 while it processes the problem through the forward channel, then produces a burst of ZebraDSL constraint declarations (`@entities`, `@domain`, `@smoothie:cherry.pet=fish`, etc.) during Phase 3. Coupling strengths show that the auxiliary attends primarily to the first presentation of each clue during Phase 2; when the primary reiterates clues in its thought block, the auxiliary registers only the first few tokens before emitting its translation. Forced output tokens (green) carry the solver's JSON solution back through the reverse channel. The primary model's output (left column) transitions from internal reasoning to the correct final JSON assignment.

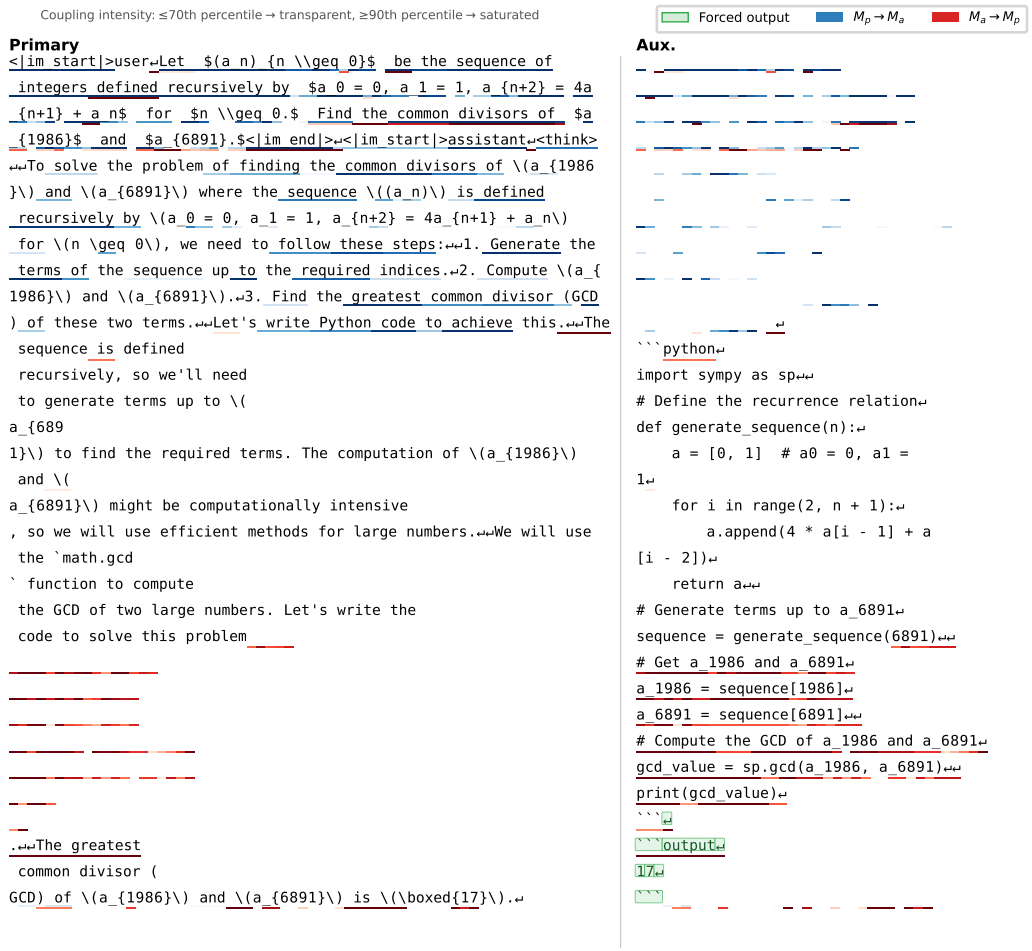


Figure 10: **Full coupling trace: GCD of recurrence sequence terms** (twin Qwen3-4B, identity interface, Python tool). The auxiliary (right) waits with spaces while the primary processes the problem, then emits a correct Python program that generates the sequence and computes $\gcd(a_{1986}, a_{6891}) = 17$. The recurrence relation, initial conditions, and both target indices were recovered entirely from hidden-state signals; the auxiliary never sees the problem text. Forward coupling (blue) is elevated during the primary's problem-processing phase. Reverse coupling (red) spikes in two contexts: while the auxiliary is still writing code, guiding the primary to emit space tokens and wait for completion; and when the primary model produces its boxed answer, at which point the result flows over from the auxiliary. Row breaks in the visualization are dictated by the auxiliary's explicit newlines for code readability; additional wrapping within rows is due to horizontal space constraints maintaining token-level synchronization.

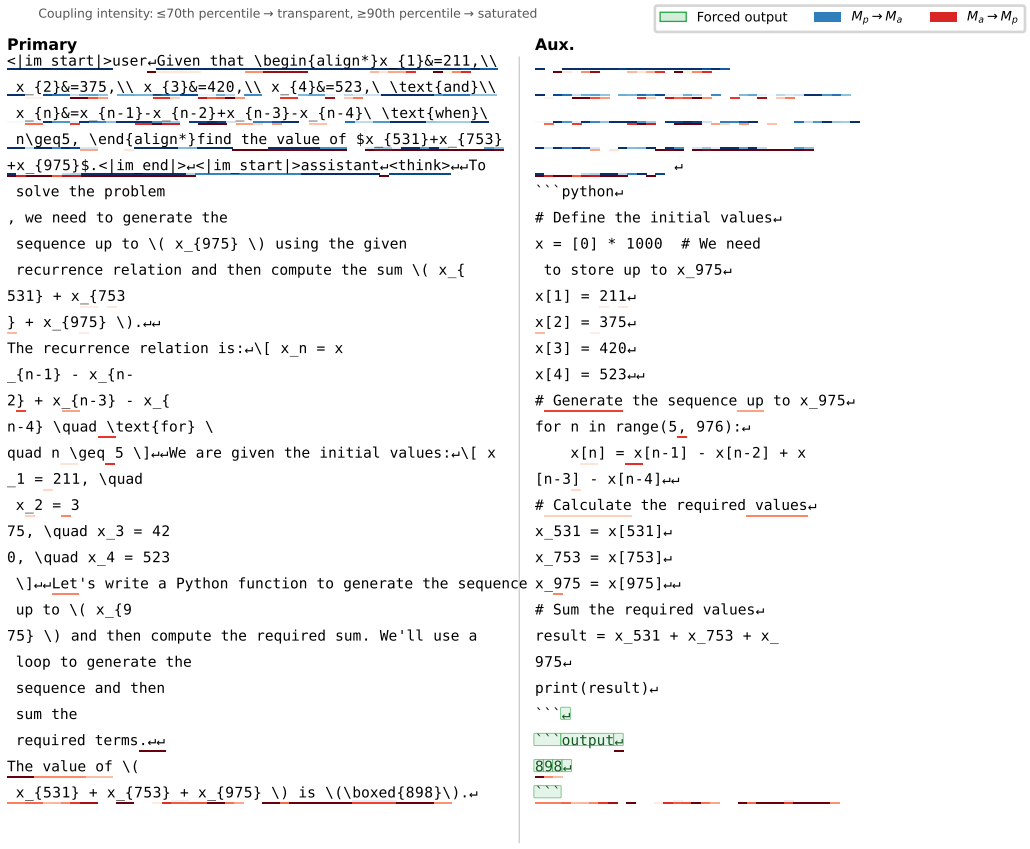


Figure 11: **Full coupling trace: recursive sequence evaluation** (twin Qwen3-4B, identity interface, Python tool). The auxiliary (right) recovers all seven parameters (4 initial values and 3 target indices) plus the recurrence structure from hidden states alone, generating correct Python code that yields 898. This example demonstrates high-bandwidth information transfer through the neural channel: 7 distinct numerical values and one structural relation are communicated without any text exchange.

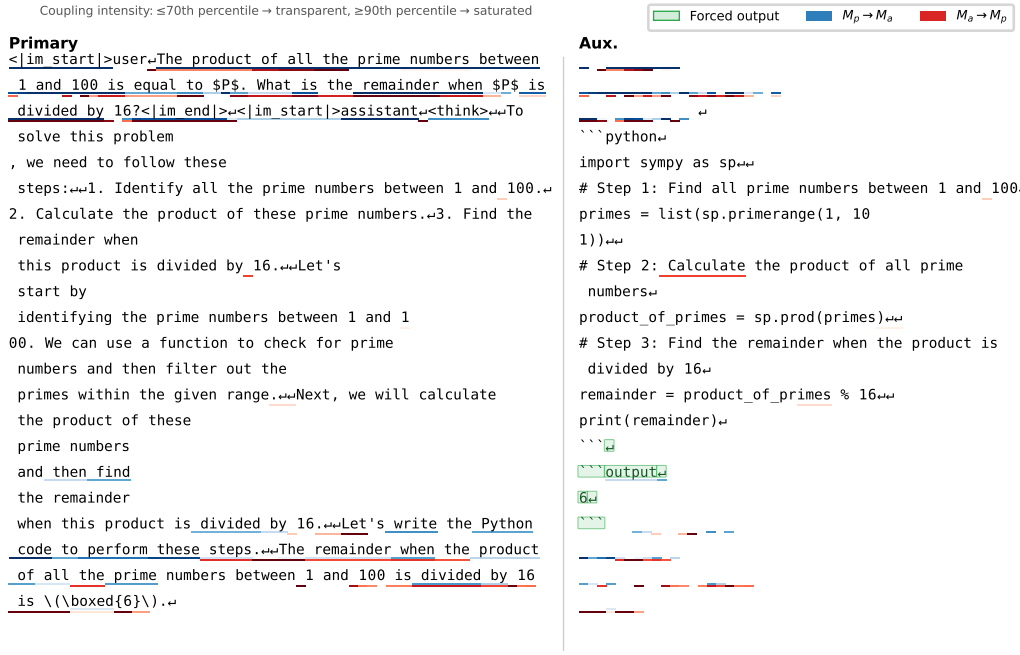


Figure 12: **Full coupling trace: prime product modular arithmetic** (twin Qwen3-4B, identity interface, Python tool). The auxiliary (right) generates a concise program using `sympy.primerange` and modular reduction, producing the correct answer of 6. Unlike the previous two examples which require recovering many numerical parameters, this problem tests whether the channel can transmit the *structural* content of a number-theoretic question (the concept of "product of primes in a range, reduced modulo 16") without any text.